PRACTICAL NO. 1

Aim: Write a program to analysis of a business dataset to identify patterns, correlations, and trends. Visualize the data using techniques such as histograms, scatter plots, and box plots to gain insights into the data distribution and relationships between variables.

Theory:

Predictive Business Analysis: Predictive Business Analysis (PBA) is the process of using historical business data, statistical techniques, and analytical methods to identify patterns and predict possible future outcomes. It helps organizations make data-driven decisions rather than relying only on assumptions or intuition. PBA is widely used for understanding customer behavior, sales performance, market trends, and business risks. Techniques such as statistical analysis, correlation, regression, forecasting, and data visualization are commonly used to extract useful information from business datasets.

Data Analysis and Preprocessing: In this experiment, a real-world business dataset is analyzed to identify important patterns, correlations, and trends. Before performing the analysis, the dataset is inspected for its structure, data types, missing values, and inconsistencies. Data preprocessing is an essential step because incomplete or incorrect data can affect the accuracy of the analysis. Missing values and irrelevant records are handled appropriately so that the cleaned dataset can be used for further statistical analysis and visualization.

Exploratory Data Analysis: Exploratory Data Analysis (EDA) is used to understand the main characteristics of a dataset through statistical summaries and graphical techniques. It helps identify the distribution of variables, relationships between attributes, trends, and unusual observations. In this experiment, EDA is performed on the Superstore sales data to understand different aspects of sales transactions. Measures such as mean, median, range, and correlation can provide useful information about the behavior and relationships present in the dataset.

Data Visualization: Data visualization represents information graphically, making large datasets easier to understand and interpret. In this experiment, different visualization techniques such as histograms, scatter plots, and box plots are used. Histograms help study the distribution of numerical variables such as sales. Scatter plots are used to examine relationships and possible correlations between numerical variables, while box plots show the spread, median, and potential outliers in the data. These visualizations help identify patterns that may not be obvious from raw data.

Business Insights: The final objective of the experiment is to extract meaningful business insights from the Superstore sales dataset. By analyzing sales data and visualizing relationships between variables, important trends and patterns can be identified. Such insights can help in understanding sales behavior, product performance, customer segments, and geographical patterns. The experiment demonstrates how Python and data visualization techniques can be applied to business data to support better interpretation, analysis, and data-driven decision-making.

Dataset Description:

The dataset used in this experiment is the Superstore Sales Forecasting dataset, obtained from Kaggle. It contains historical sales transaction data of a retail business and includes information about orders, customers, products, locations, and sales. The dataset consists of 9,800 rows and 18 columns, providing sufficient information for performing exploratory business analysis and identifying patterns and relationships within the data. The columns include attributes such as Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, City, State, Postal Code, Region, Product ID, Category, Sub-Category, Product Name, and Sales. During data preprocessing, the dataset was checked for missing values. 11 missing values were identified in the Postal Code column, corresponding to 11 records. Since these records contained missing postal-code information and were not suitable for geographical analysis, the 11 rows were dropped from the dataset. The cleaned dataset was then used for further analysis and visualization.

Source Code and Output:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
data = pd.read_csv("train.csv")
```

```
data.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales
0	1	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420.0	South	FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase	261.9600
1	2	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420.0	South	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400
2	3	CA-2017-138688	12/06/2017	16/06/2017	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036.0	West	OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Address Labels for Typewriters b...	14.6200
3	4	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311.0	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775
4	5	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311.0	South	OFF-ST-10000760	Office Supplies	Storage	Eldon Fold 'N Roll Cart System	22.3680

```
data.tail()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales
9795	9796	CA-2017-125920	21/05/2017	28/05/2017	Standard Class	SH-19975	Sally Hughsby	Corporate	United States	Chicago	Illinois	60610.0	Central	OFF-BI-10003429	Office Supplies	Binders	Cardinal HOLDit! Binder Insert Strips,Extra St...	3.798
9796	9797	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	CS-12490	Cindy Schnelling	Corporate	United States	Toledo	Ohio	43615.0	East	OFF-AR-10001374	Office Supplies	Art	BIC Brite Liner Highlighters, Chisel Tip	10.368
9797	9798	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	CS-12490	Cindy Schnelling	Corporate	United States	Toledo	Ohio	43615.0	East	TEC-PH-10004977	Technology	Phones	GE 30524EE4	235.188
9798	9799	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	CS-12490	Cindy Schnelling	Corporate	United States	Toledo	Ohio	43615.0	East	TEC-PH-10000912	Technology	Phones	Anker 24W Portable Micro USB Car Charger	26.376
9799	9800	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	CS-12490	Cindy Schnelling	Corporate	United States	Toledo	Ohio	43615.0	East	TEC-AC-10000487	Technology	Accessories	SanDisk Cruzer 4 GB USB Flash Drive	10.384

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9800 entries, 0 to 9799
Data columns (total 18 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Row ID         9800 non-null   int64
 1   Order ID       9800 non-null   object
 2   Order Date     9800 non-null   object
 3   Ship Date      9800 non-null   object
 4   Ship Mode      9800 non-null   object
 5   Customer ID    9800 non-null   object
 6   Customer Name  9800 non-null   object
 7   Segment        9800 non-null   object
 8   Country        9800 non-null   object
 9   City           9800 non-null   object
 10  State          9800 non-null   object
 11  Postal Code    9789 non-null   float64
 12  Region         9800 non-null   object
 13  Product ID     9800 non-null   object
 14  Category       9800 non-null   object
 15  Sub-Category   9800 non-null   object
 16  Product Name   9800 non-null   object
 17  Sales          9800 non-null   float64
dtypes: float64(2), int64(1), object(15)
memory usage: 1.3+ MB
```

```
data.describe()
```

	Row ID	Postal Code	Sales
count	9800.000000	9789.000000	9800.000000
mean	4900.500000	55273.322403	230.769059
std	2829.160653	32041.223413	626.651875
min	1.000000	1040.000000	0.444000
25%	2450.750000	23223.000000	17.248000
50%	4900.500000	58103.000000	54.490000
75%	7350.250000	90008.000000	210.605000
max	9800.000000	99301.000000	22638.480000

```
data.isnull()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales
0	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9795	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
9796	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
9797	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
9798	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
9799	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False

9800 rows × 18 columns

```
data.isna().sum()
```

	0
Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	11
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0

dtype: int64

```
data.duplicated().sum()
```

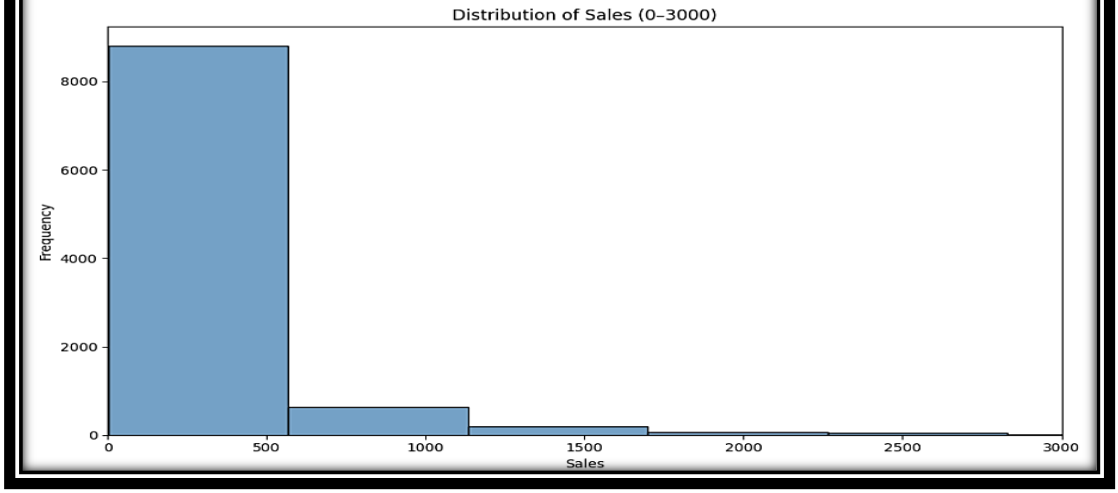
```
np.int64(0)
```

```
data.dropna()
```

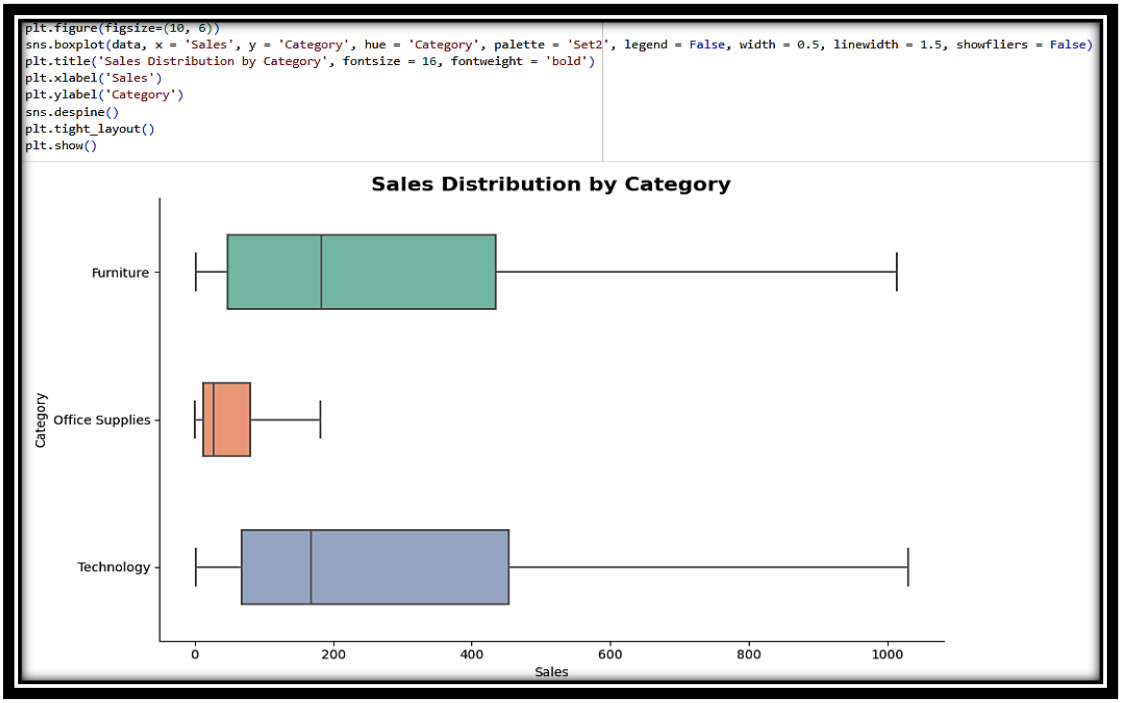
	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales
0	1	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420.0	South	FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase	261.9600
1	2	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420.0	South	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400
2	3	CA-2017-138688	12/06/2017	16/06/2017	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036.0	West	OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Address Labels for Typewriters b...	14.6200
3	4	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311.0	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775
4	5	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311.0	South	OFF-ST-10000760	Office Supplies	Storage	Eldon Fold 'N Roll Cart System	22.3680
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9795	9796	CA-2017-125920	21/05/2017	28/05/2017	Standard Class	SH-19975	Sally Hughsby	Corporate	United States	Chicago	Illinois	60610.0	Central	OFF-BI-10003429	Office Supplies	Binders	Cardinal HOLDit! Binder Insert Strips,Extra St...	3.7980
9796	9797	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	CS-12490	Cindy Schnelling	Corporate	United States	Toledo	Ohio	43615.0	East	OFF-AR-10001374	Office Supplies	Art	BIC Brite Liner Highlighters, Chisel Tip	10.3680
9797	9798	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	CS-12490	Cindy Schnelling	Corporate	United States	Toledo	Ohio	43615.0	East	TEC-PH-10004977	Technology	Phones	GE 30524EE4	235.1880
9798	9799	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	CS-12490	Cindy Schnelling	Corporate	United States	Toledo	Ohio	43615.0	East	TEC-PH-10000912	Technology	Phones	Anker 24W Portable Micro USB Car Charger	26.3760
9799	9800	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	CS-12490	Cindy Schnelling	Corporate	United States	Toledo	Ohio	43615.0	East	TEC-AC-10000487	Technology	Accessories	SanDisk Cruzer 4 GB USB Flash Drive	10.3840

9789 rows × 18 columns

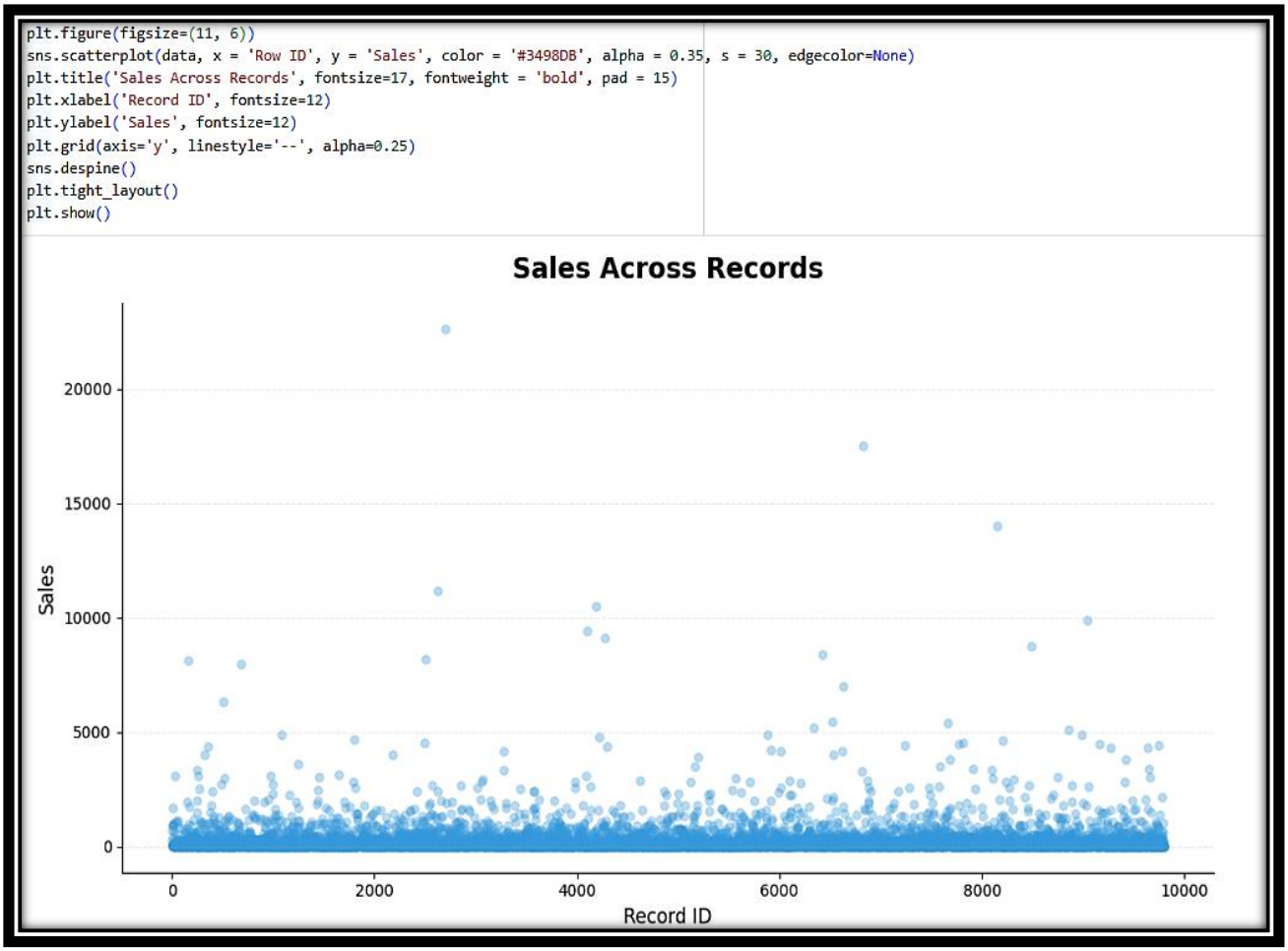
```
plt.figure(figsize=(10, 6))
sns.histplot(data, x = 'Sales', bins = 40, color = 'steelblue', edgecolor = 'black')
plt.xlim(0, 3000)
plt.title('Distribution of Sales (0-3000)')
plt.xlabel('Sales')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(10, 6))
sns.boxplot(data, x = 'Sales', y = 'Category', hue = 'Category', palette = 'Set2', legend = False, width = 0.5, linewidth = 1.5, showfliers = False)
plt.title('Sales Distribution by Category', fontsize = 16, fontweight = 'bold')
plt.xlabel('Sales')
plt.ylabel('Category')
sns.despine()
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(11, 6))
sns.scatterplot(data, x = 'Row ID', y = 'Sales', color = '#3498DB', alpha = 0.35, s = 30, edgecolor=None)
plt.title('Sales Across Records', fontsize=17, fontweight = 'bold', pad = 15)
plt.xlabel('Record ID', fontsize=12)
plt.ylabel('Sales', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.25)
sns.despine()
plt.tight_layout()
plt.show()
```



Learning Outcome:

PRACTICAL NO. 2

Aim: Build a regression model to predict a business metric, such as sales or customer churn, based on relevant features.

Theory:

Predictive Business Analytics (PBA): Predictive Business Analytics (PBA) is the application of statistical techniques, machine learning algorithms, and historical business data to predict future outcomes and support decision-making. Unlike descriptive analytics, which explains what has already happened, and diagnostic analytics, which investigates why it happened, predictive analytics focuses on what is likely to happen next. PBA helps organizations forecast demand, estimate revenue, identify customer behavior, manage risks, and optimize resources. In this experiment, PBA is applied to historical Superstore sales data to identify relationships between sales and relevant business features. A regression model is then developed to estimate sales for new observations, demonstrating how predictive techniques can convert historical transaction data into actionable business insights.

Business Metric: A business metric is a measurable quantity used to evaluate the performance, efficiency, or financial health of a business. Common metrics include sales revenue, profit, customer churn rate, conversion rate, and average order value. In this experiment, Sales is selected as the target business metric because it represents the revenue generated from individual transactions. The objective is to predict sales using relevant customer, product, geographical, and order-related features such as Ship Mode, Segment, Region, State, City, Category, Sub-Category, Product Name, order year, order month, and shipping duration. Accurate sales prediction can support demand forecasting, inventory planning, product management, and revenue estimation.

Random Forest Regression Model: Random Forest Regression is a supervised machine-learning technique used to predict a continuous target variable by combining the predictions of multiple decision trees. Instead of relying on a single decision tree, Random Forest constructs several trees using different subsets of the training data and features. The final prediction is obtained by averaging the predictions from all the individual trees. This ensemble approach generally improves prediction accuracy and reduces the risk of overfitting compared with a single decision tree. In this experiment, a Random Forest Regressor with 100 decision trees is used to predict Sales. The model is trained on 80% of the data and evaluated on the remaining 20% test data. The random_state parameter ensures reproducible results, while n_jobs=-1 allows all available processor cores to be used for faster training.

Dataset Features: The Superstore dataset contains a mixture of numerical, categorical, identifier, and date-based features. Relevant predictors include Quantity, Discount (where available), Profit, Ship Mode, Segment, Region, State, City, Category, and Sub-Category. Order Date can be transformed into year, month, and day-of-week features to capture temporal patterns. Identifier fields such as Row ID, Order ID, Customer ID, and Product ID are generally excluded because they identify records rather than provide meaningful predictive information. Product Name and Customer Name can also be excluded because they create very high-cardinality categorical variables. Feature selection is important because irrelevant or redundant variables can increase model complexity and reduce interpretability.

Dataset Description:

The Kaggle dataset provided for this experiment is the Superstore Sales Forecasting dataset. It represents transactional sales information from a Superstore and is designed for sales analysis and forecasting applications. The data contains order and shipping information, customer details, geographical attributes, product categories, quantities, sales, and profit-related information. Each observation represents a sales transaction or product-level order record. The dataset is particularly useful for predictive business analytics because it combines numerical and categorical variables with date information, allowing relationships between business activities and sales performance to be modeled. For this experiment, the dataset is loaded, cleaned, transformed, divided into training and testing data, and used to build a regression model for predicting Sales.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("/content/exp_2_train.csv")

df["Order Date"] = pd.to_datetime(df["Order Date"], dayfirst=True)
df["Ship Date"] = pd.to_datetime(df["Ship Date"], dayfirst=True)
df["Year"] = df["Order Date"].dt.year
df["Month"] = df["Order Date"].dt.month
df["Ship Days"] = (df["Ship Date"] - df["Order Date"]).dt.days

features = ["Ship Mode", "Segment", "Region", "State", "City",
    "Category", "Sub-Category", "Product Name",
    "Year", "Month", "Ship Days"]

X = df[features]
y = df["Sales"]

categorical = ["Ship Mode", "Segment", "Region", "State", "City",
    "Category", "Sub-Category", "Product Name"]

preprocessor = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical)
], remainder="passthrough")
```

```
X = preprocessor.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = RandomForestRegressor(n_estimators=100,
    random_state=42,
    n_jobs=-1)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("MAE :", round(mae, 2))
print("RMSE:", round(rmse, 2))
print("R²  :", round(r2, 3))

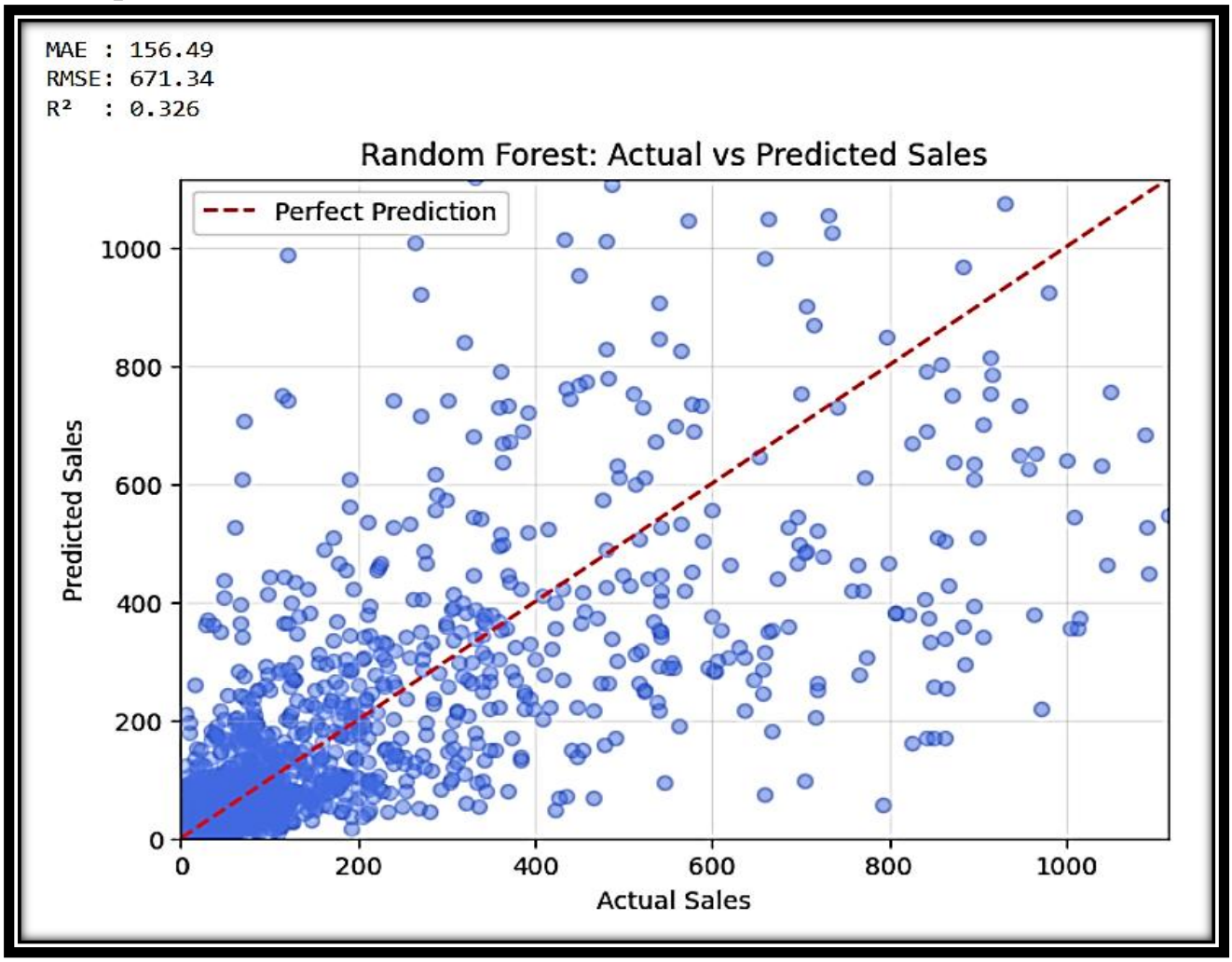
limit = y_test.quantile(0.95)
plt.figure(figsize=(7, 5))
plt.scatter(y_test[y_test <= limit],
    y_pred[y_test <= limit],
    color="royalblue",
    alpha=0.5)

plt.plot([0, limit], [0, limit],
    color="red",
    linestyle="--",
    label="Perfect Prediction")

plt.xlim(0, limit)
plt.ylim(0, limit)

plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Random Forest: Actual vs Predicted Sales")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Output:



Learning Outcome:

PRACTICAL NO. 3

Aim: Write a program to apply different regression algorithms, such as linear regression, decision trees, or random forests, and compare their performance using evaluation metrics like Mean Squared Error (MSE) or R-squared.

Theory:

Predictive Business Analytics (PBA): Predictive Business Analytics (PBA) uses historical and current data to predict future outcomes and support better business decisions. It combines statistical techniques, machine learning, data preprocessing, and evaluation methods to identify patterns in business data. In retail, PBA can be used for sales forecasting, demand estimation, product analysis, and customer behaviour analysis. Regression is an important PBA technique for predicting continuous numerical values. In this experiment, PBA is applied to the Superstore Sales dataset to predict Sales using different regression algorithms. Their performance is compared using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-squared (R^2) metrics.

Regression Models: Regression is a supervised machine learning technique used to identify relationships between independent variables and a continuous dependent variable. It learns patterns from existing data and uses them to predict numerical outcomes for new data. In this experiment, Sales is the target variable, while Ship Mode, Segment, Region, State, City, Category, Sub-Category, Product Name, Year, Month, and Ship Days are predictor variables. Three algorithms are used: Linear Regression, Decision Tree Regression, and Random Forest Regression. Comparing different models helps identify which algorithm provides better predictions for the given dataset.

Algorithms Used: Linear Regression assumes a linear relationship between input and target variables and finds the best-fitting equation by minimizing errors. Decision Tree Regression divides data into smaller groups using a tree structure and can capture nonlinear relationships. Random Forest Regression combines multiple decision trees to produce more robust predictions and generally reduces overfitting. In this experiment, all three algorithms are trained and tested on the same data so their performance can be compared fairly.

Dataset Features and Preprocessing: The Superstore dataset contains categorical and numerical retail information. Categorical features are converted into numerical form using One-Hot Encoding. The categorical features used are Ship Mode, Segment, Region, State, City, Category, Sub-Category, and Product Name. Order Date and Ship Date are converted into datetime format. Year and Month are extracted from Order Date, while Ship Days is calculated as the difference between Ship Date and Order Date. These features provide information about orders, products, customers, and shipping. Sales is selected as the target variable because it is a continuous numerical business metric.

Model Evaluation Metrics: The models are evaluated using MSE, RMSE, and R^2. MSE measures the average squared difference between actual and predicted values, where a lower value indicates better performance. RMSE is the square root of MSE and represents error in the same unit as the target variable. R^2 measures how much variation in the target variable is explained by the model, with a higher value generally indicating better performance.

Dataset Description:

The Superstore Sales Forecasting dataset is a retail sales dataset containing historical information about customer orders, products, shipping, geographical regions, and sales performance. It includes fields such as Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer information, Segment, Country, City, State, Postal Code, Region, Product information, Category, Sub-Category, Product Name, and Sales. The dataset is suitable for predictive analytics because it contains multiple attributes that can be used to understand and predict business performance. In this experiment, Sales is selected as the dependent variable or target attribute. The predictor variables include Ship Mode, Segment, Region, State, City, Category, Sub-Category, Product Name, Year, Month, and Ship Days. Categorical features are transformed using One-Hot Encoding, while date fields are processed to derive useful numerical features. The dataset is divided into training and testing sets to evaluate the regression models on unseen observations.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
df = pd.read_csv("/content/exp_3_train.csv")
df["Order Date"] = pd.to_datetime(df["Order Date"], dayfirst=True)
df["Ship Date"] = pd.to_datetime(df["Ship Date"], dayfirst=True)
df["Year"] = df["Order Date"].dt.year
df["Month"] = df["Order Date"].dt.month
df["Ship Days"] = (df["Ship Date"] - df["Order Date"]).dt.days
df = df.dropna()
features = [
    "Ship Mode",
    "Segment",
    "Region",
    "State",
    "City",
    "Category",
    "Sub-Category",
    "Product Name",
    "Year",
    "Month",
    "Ship Days"]
X = df[features]
y = df["Sales"]
```

```
categorical = [
    "Ship Mode",
    "Segment",
    "Region",
    "State",
    "City",
    "Category",
    "Sub-Category",
    "Product Name"]
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical)],
    remainder="passthrough")
X = preprocessor.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42)
print("Training samples:", X_train.shape[0])
print("Testing samples :", X_test.shape[0])
print()
models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(
        random_state=42,
        max_depth=20),
    "Random Forest": RandomForestRegressor(
        n_estimators=100,
        random_state=42,
        n_jobs=-1)}
results = []
predictions = {}
for name, model in models.items():
    print("Training:", name)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)
    results.append({
        "Model": name,
        "MSE": mse,
        "RMSE": rmse,
        "R2 Score": r2})
    predictions[name] = y_pred
results_df = pd.DataFrame(results)
print("\n================ MODEL PERFORMANCE ================\n")
print(results_df.round(3).to_string(index=False))
```

```
best_r2 = results_df.loc[results_df["R2 Score"].idxmax()]
best_mse = results_df.loc[results_df["MSE"].idxmin()]
print("\n====================================================")
print("Best Model based on R²:")
print(best_r2["Model"], "-> R² =", round(best_r2["R2 Score"], 3))
print("\nBest Model based on MSE:")
print(best_mse["Model"], "-> MSE =", round(best_mse["MSE"], 3))
print("====================================================")
plt.figure(figsize=(8, 5))
plt.bar(
    results_df["Model"],
    results_df["R2 Score"])
plt.xlabel("Regression Model")
plt.ylabel("R² Score")
plt.title("Comparison of Regression Models using R² Score")
plt.ylim(
    min(results_df["R2 Score"]) - 0.05,
    min(1.0, max(results_df["R2 Score"]) + 0.05))
plt.grid(axis="y", alpha=0.3)
plt.xticks(rotation=15)
plt.show()
plt.figure(figsize=(8, 5))
plt.bar(
    results_df["Model"],
    results_df["MSE"])
plt.xlabel("Regression Model")
plt.ylabel("Mean Squared Error (MSE)")
plt.title("Comparison of Regression Models using MSE")
plt.grid(axis="y", alpha=0.3)
plt.xticks(rotation=15)
plt.show()
best_model_name = best_r2["Model"]
best_predictions = predictions[best_model_name]
limit = y_test.quantile(0.95)
plt.figure(figsize=(7, 5))
plt.scatter(
    y_test[y_test <= limit],
    best_predictions[y_test <= limit],
    alpha=0.5)
plt.plot(
    [0, limit],
    [0, limit],
    linestyle="--",
    label="Perfect Prediction")
plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title(f"{best_model_name}: Actual vs Predicted Sales")
plt.xlim(0, limit)
plt.ylim(0, limit)
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Output:

```
Training samples: 7831
Testing samples : 1958

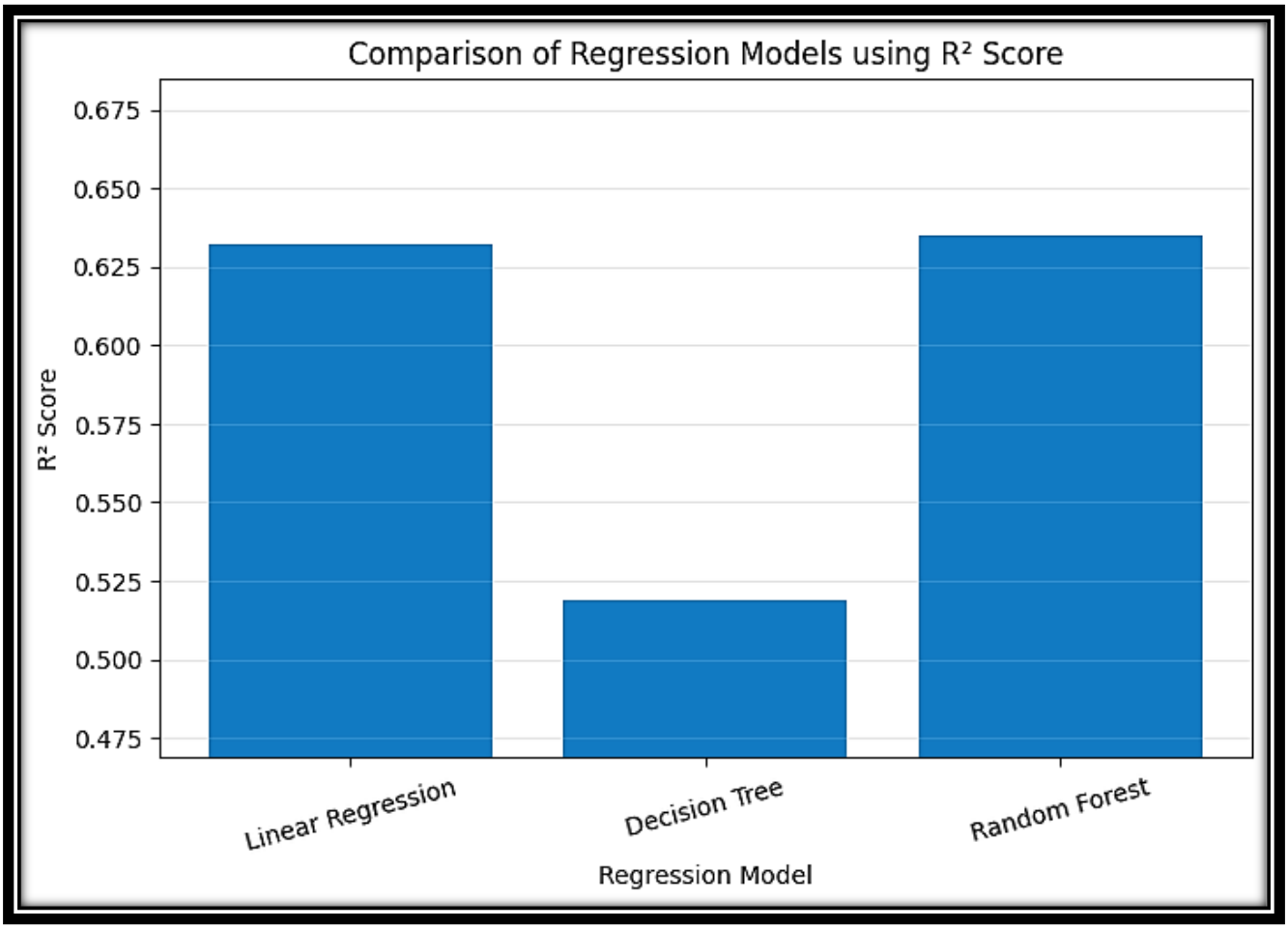
Training: Linear Regression
Training: Decision Tree
Training: Random Forest

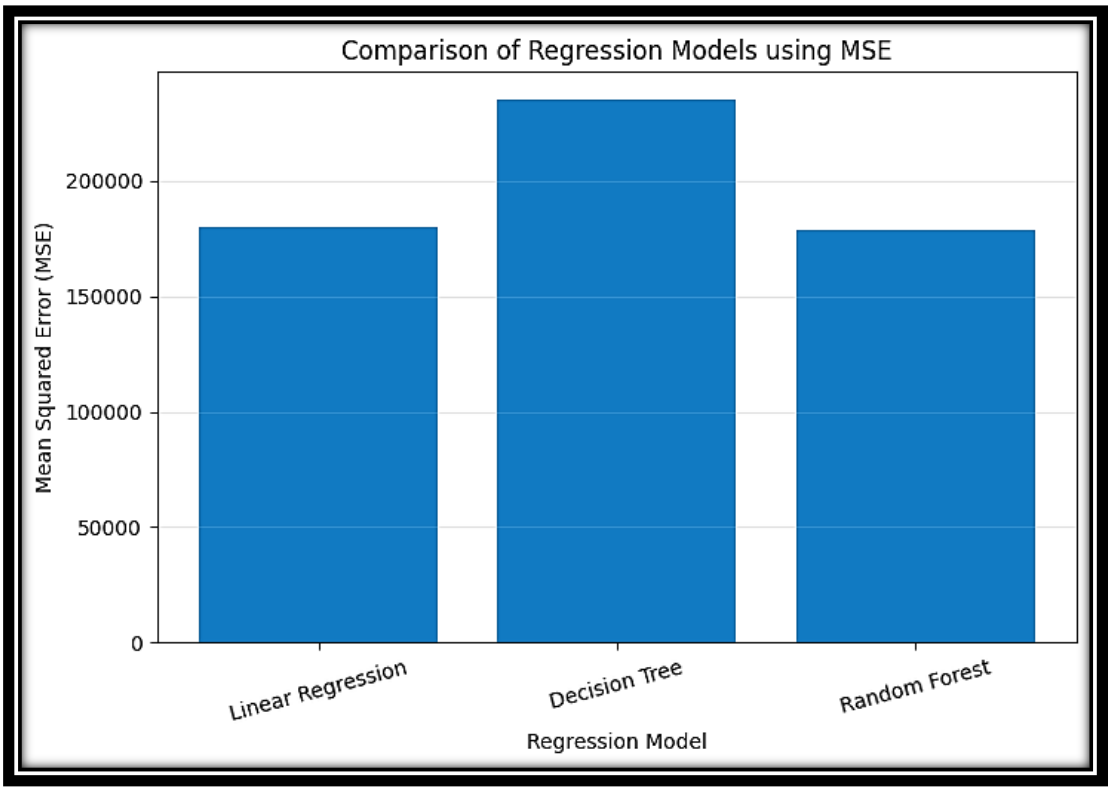
================ MODEL PERFORMANCE ================

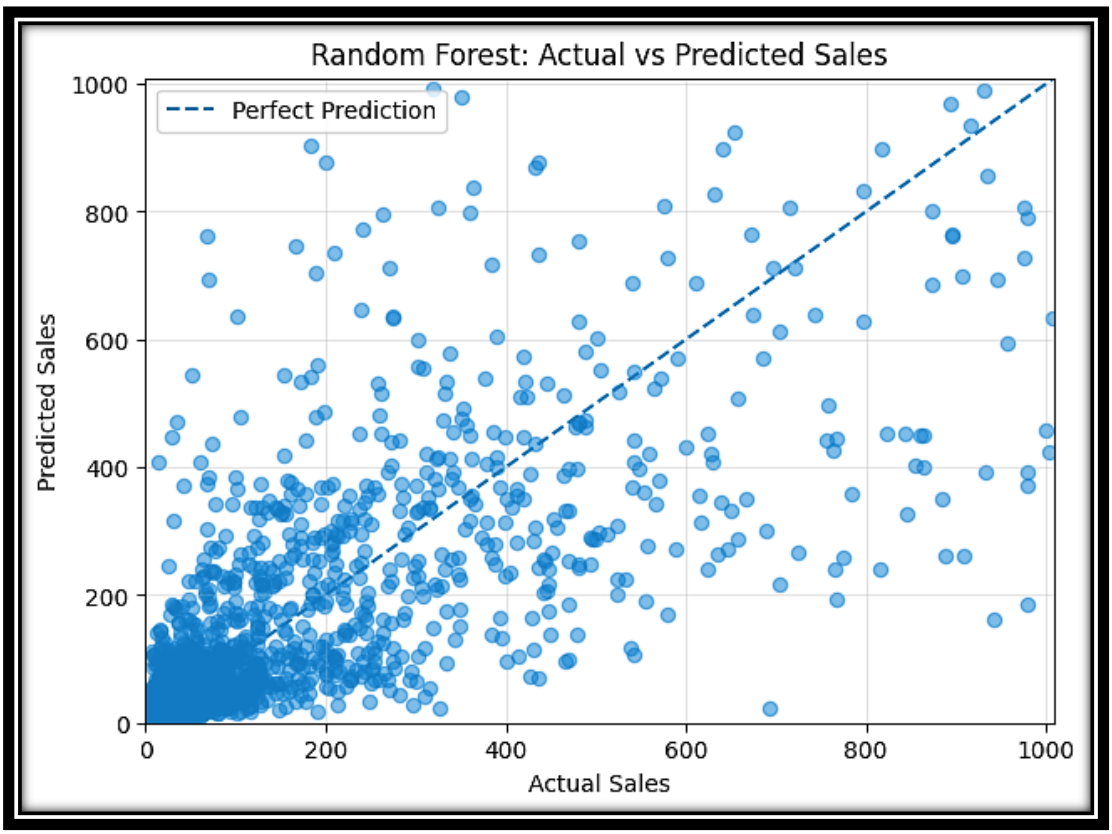
             Model        MSE    RMSE  R2 Score
Linear Regression 179873.357 424.115     0.632
    Decision Tree 235419.065 485.200     0.519
    Random Forest 178596.334 422.607     0.635

=====================================================
Best Model based on R²:
Random Forest -> R² = 0.635

Best Model based on MSE:
Random Forest -> MSE = 178596.334
=====================================================
```







Learning Outcome:

PRACTICAL NO. 4

Aim: Perform a classification analysis to predict a binary or multi-class business outcome, such as customer segmentation or product categorization. Experiment with classification algorithms like logistic regression, support vector machines (SVM), or decision trees. Evaluate the models using metrics such as accuracy, precision, recall, and F1-score.

Theory:

Predictive Business Analytics (PBA): Predictive Business Analytics (PBA) uses historical data, statistical methods, and machine learning techniques to predict future outcomes and support business decision-making. It helps organizations identify patterns, estimate outcomes, classify customers or products, and improve business strategies. Classification is an important machine learning technique used in PBA when the output belongs to predefined categories. For example, businesses can classify customers into different segments or categorize products based on their characteristics. In this experiment, classification concepts are applied to the Iris dataset to predict the species of a flower. Three classification algorithms, namely Logistic Regression, Support Vector Machine (SVM), and Decision Tree, are applied and compared using Accuracy, Precision, Recall, and F1-Score.

Classification and Types: Classification is a supervised machine learning technique that assigns data observations to predefined categories or classes. The model learns relationships between input features and known class labels from training data and uses them to classify new observations. Classification can be binary or multi-class. Binary classification involves only two possible classes, such as Yes/No or Fraud/Not Fraud. Multi-class classification involves three or more classes, such as different product categories or customer groups. The Iris dataset represents a multi-class classification problem because its flowers belong to three species: Setosa, Versicolor, and Virginica. The objective of this experiment is to learn the relationship between flower measurements and species and correctly classify unseen flower observations.

Classification Algorithms: Three classification algorithms are used in this experiment: Logistic Regression, Support Vector Machine (SVM), and Decision Tree. Logistic Regression estimates the probability of an observation belonging to a particular class and is commonly used for classification problems. SVM finds an optimal boundary, or hyperplane, that separates different classes while maximizing the margin between them. Decision Tree uses a tree-like structure and makes decisions by splitting data according to feature values. These algorithms use different approaches to classification, making them suitable for comparison. All three models are trained and tested using the same Iris dataset so that their classification performance can be evaluated fairly.

Customer Segmentation and Product Categorization: Classification has several applications in business analytics, particularly in customer segmentation and product categorization. In customer segmentation, classification algorithms can assign customers to predefined groups based on characteristics such as purchasing behaviour, demographics, or spending patterns. In product categorization, algorithms can classify products into categories using features such as price, specifications, or sales information. Although the Iris dataset is

not a business dataset, its classification structure demonstrates the same fundamental machine learning process. In this experiment, the flower species act as the target categories, while flower measurements are used as predictor features. The same classification concepts can therefore be applied to real-world business problems involving predefined categories.

Dataset Features and Model Evaluation: The Iris dataset contains four numerical features: Sepal Length, Sepal Width, Petal Length, and Petal Width. These measurements are used as input features, while the flower species is the target variable. The target contains three classes: Setosa, Versicolor, and Virginica. Feature scaling using StandardScaler is applied before training the models because algorithms such as Logistic Regression and SVM can benefit from features being on a similar scale. The models are evaluated using Accuracy, Precision, Recall, and F1-Score. Accuracy measures overall correct predictions, Precision measures the correctness of positive predictions, Recall measures correctly identified instances, and F1-Score provides a balance between Precision and Recall.

Dataset Description:

The Iris dataset is a well-known dataset used for demonstrating classification and machine learning techniques. It contains 150 flower observations belonging to three species: Iris Setosa, Iris Versicolor, and Iris Virginica. Each observation contains four numerical measurements: Sepal Length, Sepal Width, Petal Length, and Petal Width. These measurements describe different physical characteristics of the flowers and are used as input features for classification. The target variable represents the species of each flower and contains three classes, making this a multi-class classification problem. In this experiment, the dataset is divided into training and testing subsets using an 80:20 ratio. The numerical features are standardized using StandardScaler before applying the classification algorithms. Logistic Regression, Support Vector Machine, and Decision Tree classifiers are trained using the processed data. Their predictions are evaluated using Accuracy, Precision, Recall, and F1-Score to determine which algorithm provides the best classification performance on the Iris dataset.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report)
```

```
iris = load_iris()
X = pd.DataFrame(
    iris.data,
    columns=iris.feature_names)
y = iris.target
print("Dataset Shape:", X.shape)
print("\nFeatures:")
print(X.head())
print("\nClasses:", iris.target_names)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
models = {
    "Logistic Regression": LogisticRegression(
        max_iter=1000,
        random_state=42),
    "SVM": SVC(
        kernel="rbf",
        random_state=42),
    "Decision Tree": DecisionTreeClassifier(
        random_state=42,
        max_depth=5)}
results = []
predictions = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(
        y_test, y_pred, average="weighted")
    recall = recall_score(
        y_test, y_pred, average="weighted")
    f1 = f1_score(
        y_test, y_pred, average="weighted")
    results.append({
```

```
    results.append({
        "Model": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1})
    predictions[name] = y_pred
results_df = pd.DataFrame(results)
print("\n================ MODEL PERFORMANCE ================\n")
print(
    results_df.round(3).to_string(index=False))
for name, y_pred in predictions.items():
    print("\n=====================================================")
    print(name)
    print("=====================================================")
    print(
        classification_report(
            y_test,
            y_pred,
            target_names=iris.target_names))
best_model = results_df.loc[
    results_df["F1-Score"].idxmax()]
print("\n================ BEST MODEL ================")
print("Model     :", best_model["Model"])
print("Accuracy  :", round(best_model["Accuracy"], 3))
print("Precision :", round(best_model["Precision"], 3))
print("Recall    :", round(best_model["Recall"], 3))
print("F1-Score  :", round(best_model["F1-Score"], 3))
print("============================================")
metrics = ["Accuracy", "Precision", "Recall", "F1-Score"]
results_plot = results_df.set_index("Model")[metrics]
results_plot.plot(
    kind="bar",
    figsize=(10, 6))
plt.title("Comparison of Classification Models")
plt.xlabel("Classification Algorithm")
plt.ylabel("Score")
plt.ylim(0, 1.1)
plt.xticks(rotation=15)
plt.legend(title="Metrics")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:

```
Dataset Shape: (150, 4)

Features:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

Classes: ['setosa' 'versicolor' 'virginica']

================ MODEL PERFORMANCE ================

              Model  Accuracy  Precision  Recall  F1-Score
Logistic Regression     0.933      0.933   0.933     0.933
                SVM     0.967      0.970   0.967     0.967
      Decision Tree     0.933      0.933   0.933     0.933

======================================================
Logistic Regression
======================================================
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10

    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30


======================================================
SVM
======================================================
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.90      0.95        10
   virginica       0.91      1.00      0.95        10

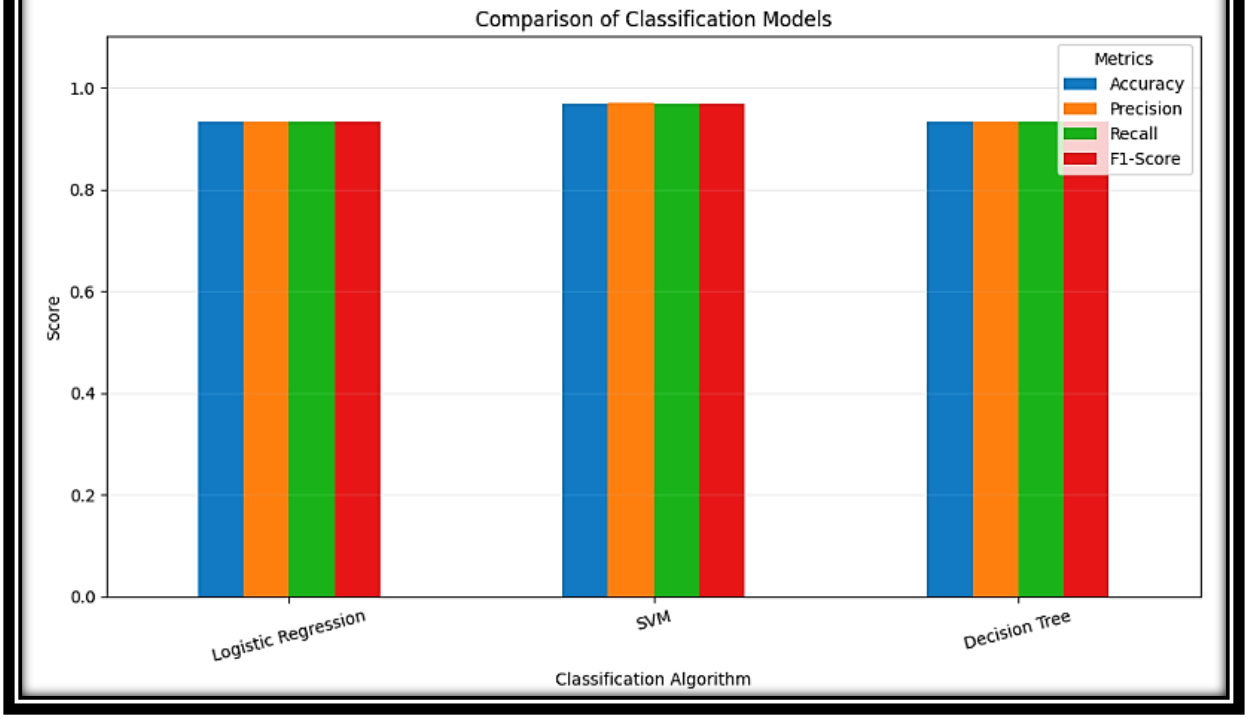
    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30
```

```
==================================================
Decision Tree
==================================================
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10

    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30


================ BEST MODEL =================
Model     : SVM
Accuracy  : 0.967
Precision : 0.97
Recall    : 0.967
F1-Score  : 0.967
==============================================
```



Learning Outcome:

PRACTICAL NO. 5

Aim: To apply time series forecasting techniques to predict future business trends or product/service demand.

Theory:

Predictive Business Analytics (PBA): Predictive Business Analytics (PBA) uses historical data, statistical techniques, and machine learning methods to predict future outcomes and support business decision-making. It helps organizations identify trends, estimate demand, forecast sales, and plan resources more effectively. In retail businesses, predictive analytics can help determine expected product demand and future sales. Time series forecasting is an important technique within PBA because business data is often collected over time. In this experiment, PBA concepts are applied to the Superstore Sales dataset to study historical sales patterns and predict future sales. Daily sales are extracted from the dataset and used to build forecasting models. The predicted values can help businesses make better decisions regarding inventory, staffing, sales planning, and resource allocation.

Time Series Forecasting: Time series forecasting is a technique used to predict future values based on observations recorded over time. Unlike ordinary machine learning problems, time series data has a chronological order, meaning past observations can influence future values. Important patterns include trend, seasonality, and fluctuations. Trend represents the long-term direction of the data, while seasonality represents repeated patterns over specific time periods. In this experiment, the Order Date is used as the time variable and Sales is aggregated on a daily basis. The resulting daily sales series is arranged chronologically and used for forecasting. Two techniques, 7-Day Moving Average and Holt-Winters Exponential Smoothing, are applied to identify patterns and estimate future sales values. The Kaggle dataset is specifically intended for sales time-series forecasting.

Forecasting Business Demand and Techniques: Time series forecasting helps predict future business trends and product or service demand, supporting inventory management, purchasing, staffing, budgeting, and marketing decisions. Accurate forecasts help businesses prepare for increased demand and avoid unnecessary stock accumulation during low-demand periods. The Superstore dataset contains several years of retail sales information, making it suitable for analyzing historical patterns and predicting future demand. In this experiment, daily Sales values are used to forecast the following seven days. Two techniques are applied: 7-Day Moving Average and Holt-Winters Exponential Smoothing. Moving Average uses the average sales of the previous seven days, while Holt-Winters models trend and weekly seasonality. Both models are evaluated using MAE and RMSE, where lower values indicate better performance.

Dataset Features and Preprocessing: The Superstore Sales dataset contains information about retail orders, including Order Date, Ship Date, Ship Mode, customer details, location, product category, sub-category, and Sales. For this experiment, Order Date and Sales are the

primary features used for time series forecasting. Order Date acts as the time index, while Sales represents the business value being forecast. Individual order-level Sales values are grouped by Order Date and summed to obtain total daily sales. Missing or invalid dates are removed, and a continuous daily date range is created so that the time series contains observations for each day. The data is then divided chronologically into training and testing periods. The final model uses the complete historical series to forecast sales for the next seven days.

Dataset Description:

The Superstore Sales Forecasting dataset is a retail dataset containing historical order information from a global superstore over multiple years. The dataset includes details related to orders, customers, products, geographical locations, shipping, and sales. Important fields include Order Date, Ship Date, Ship Mode, Segment, Region, State, City, Category, Sub-Category, Product Name, and Sales. The dataset is particularly suitable for time series forecasting because the orders contain dates that allow sales patterns to be arranged chronologically. In this experiment, Order Date is used as the time variable and Sales is selected as the forecasting target. Individual transactions are grouped by Order Date and their Sales values are summed to calculate total daily sales. The resulting daily time series is used to identify historical patterns and forecast future sales. The Kaggle dataset describes its purpose as performing time series analysis and predicting sales for the next seven days.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import glob
import warnings
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from sklearn.metrics import mean_absolute_error, mean_squared_error
warnings.filterwarnings("ignore")

files = glob.glob("/content/*.csv")

if len(files) == 0:
    raise FileNotFoundError("No CSV file found in /content/")

file_path = files[0]
df = pd.read_csv(file_path)

print("File used:", file_path)
print("Dataset Shape:", df.shape)

df["Order Date"] = pd.to_datetime(
    df["Order Date"],
    dayfirst=True,
    errors="coerce")
```

```
df = df.dropna(subset=["Order Date", "Sales"])

daily_sales = (
    df.groupby("Order Date")["Sales"]
      .sum()
      .sort_index())

full_dates = pd.date_range(
    start=daily_sales.index.min(),
    end=daily_sales.index.max(),
    freq="D")

daily_sales = daily_sales.reindex(
    full_dates,
    fill_value=0)

daily_sales.index.name = "Date"

print("\nTime Series Information")
print("-----------------------")
print("Start Date:", daily_sales.index.min().date())
print("End Date  :", daily_sales.index.max().date())
print("Number of Days:", len(daily_sales))

plt.figure(figsize=(12, 5))

plt.plot(
    daily_sales.index,
    daily_sales.values)

plt.title("Daily Superstore Sales")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

```
test_days = 7

train = daily_sales.iloc[:-test_days]
test = daily_sales.iloc[-test_days:]

print("\nTraining Days:", len(train))
print("Testing Days :", len(test))

window = 7

moving_average_value = train.tail(window).mean()

ma_forecast = np.repeat(
    moving_average_value,
    test_days)

hw_model = ExponentialSmoothing(
    train,
    trend="add",
    seasonal="add",
    seasonal_periods=7)

hw_fit = hw_model.fit(
    optimized=True)

hw_forecast = hw_fit.forecast(test_days)

mae_ma = mean_absolute_error(
    test,
    ma_forecast)

rmse_ma = np.sqrt(
    mean_squared_error(
        test,
        ma_forecast))
```

```
mae_hw = mean_absolute_error(
    test,
    hw_forecast)

rmse_hw = np.sqrt(
    mean_squared_error(
        test,
        hw_forecast))

results = pd.DataFrame({
    "Model": [
        "7-Day Moving Average",
        "Holt-Winters"],
    "MAE": [
        mae_ma,
        mae_hw],
    "RMSE": [
        rmse_ma,
        rmse_hw]})

print("\n================ MODEL PERFORMANCE ================\n")
print(results.round(2).to_string(index=False))

best_model = results.loc[
    results["RMSE"].idxmin(),
    "Model"]

print("Best Forecasting Model:", best_model)

plt.figure(figsize=(12, 5))

plt.plot(
    test.index,
    test.values,
    marker="o",
    label="Actual Sales")

plt.plot(
    test.index,
    ma_forecast,
    marker="o",
    linestyle="--",
    label="Moving Average")
```

```
plt.plot(
    test.index,
    hw_forecast,
    marker="o",
    linestyle="--",
    label="Holt-Winters")

plt.title("Actual vs Forecasted Sales")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

final_model = ExponentialSmoothing(
    daily_sales,
    trend="add",
    seasonal="add",
    seasonal_periods=7)

final_fit = final_model.fit(
    optimized=True)

future_forecast = final_fit.forecast(7)

forecast_dates = pd.date_range(
    start=daily_sales.index[-1] + pd.Timedelta(days=1),
    periods=7,
    freq="D")

forecast_df = pd.DataFrame({
    "Date": forecast_dates,
    "Forecasted Sales": future_forecast.values})

print("\n================ NEXT 7 DAYS FORECAST ================\n")
print(
    forecast_df.round(2).to_string(index=False))

plt.figure(figsize=(12, 5))
```

```
plt.plot(
    daily_sales.tail(30).index,
    daily_sales.tail(30).values,
    marker="o",
    label="Historical Sales")

plt.plot(
    forecast_dates,
    future_forecast.values,
    marker="o",
    linestyle="--",
    label="7-Day Forecast")

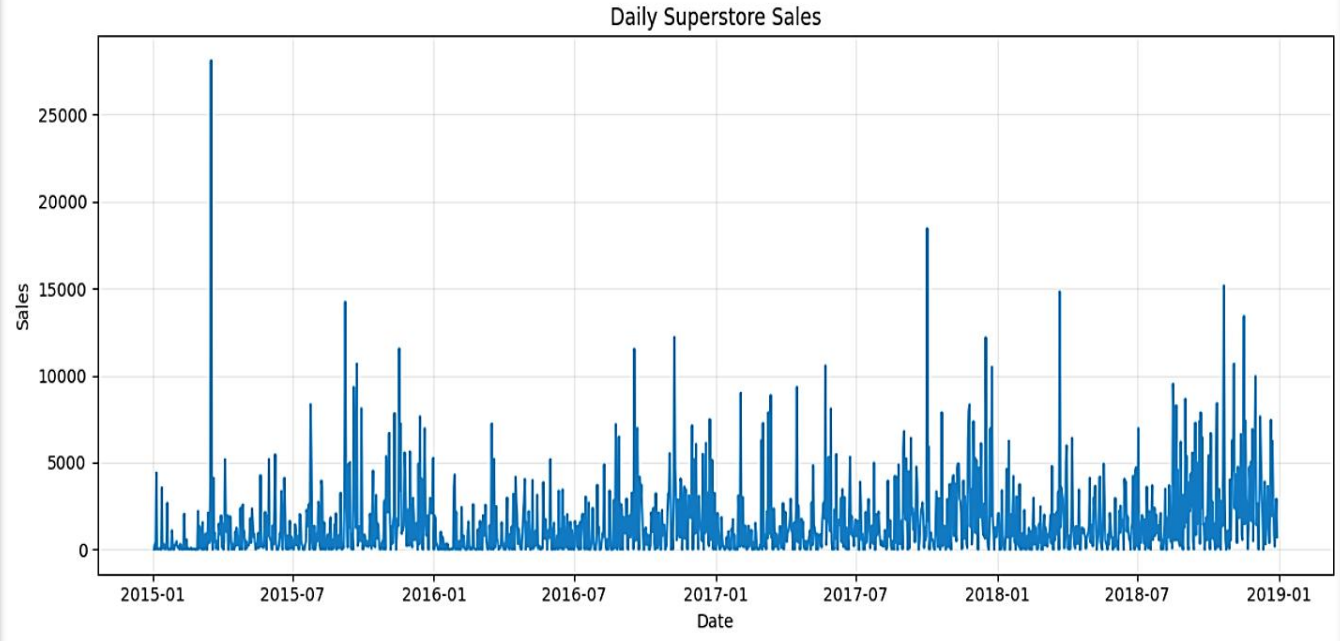
plt.axvline(
    daily_sales.index[-1],
    linestyle=":",
    label="Forecast Start")

plt.title("Superstore Sales: Next 7 Days Forecast")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:

```
File used: /content/SuperStore_Sales_Dataset.csv
Dataset Shape: (9800, 18)

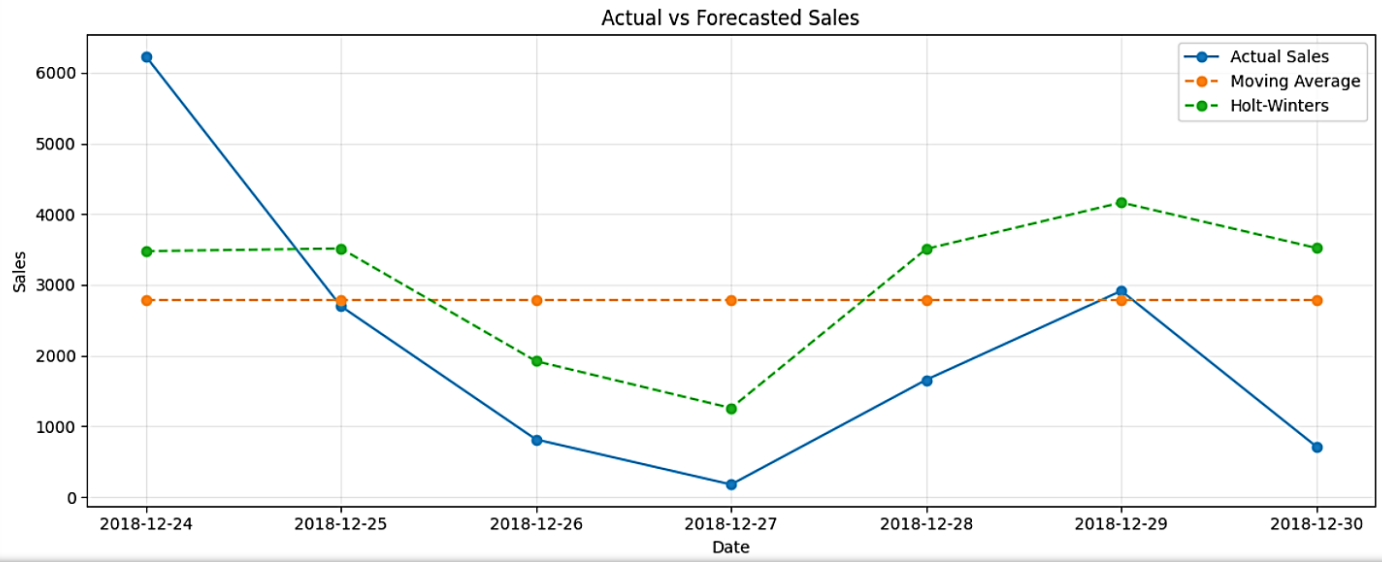
Time Series Information
-----------------------
Start Date: 2015-01-03
End Date  : 2018-12-30
Number of Days: 1458
```




```
Training Days: 1451
Testing Days : 7
```

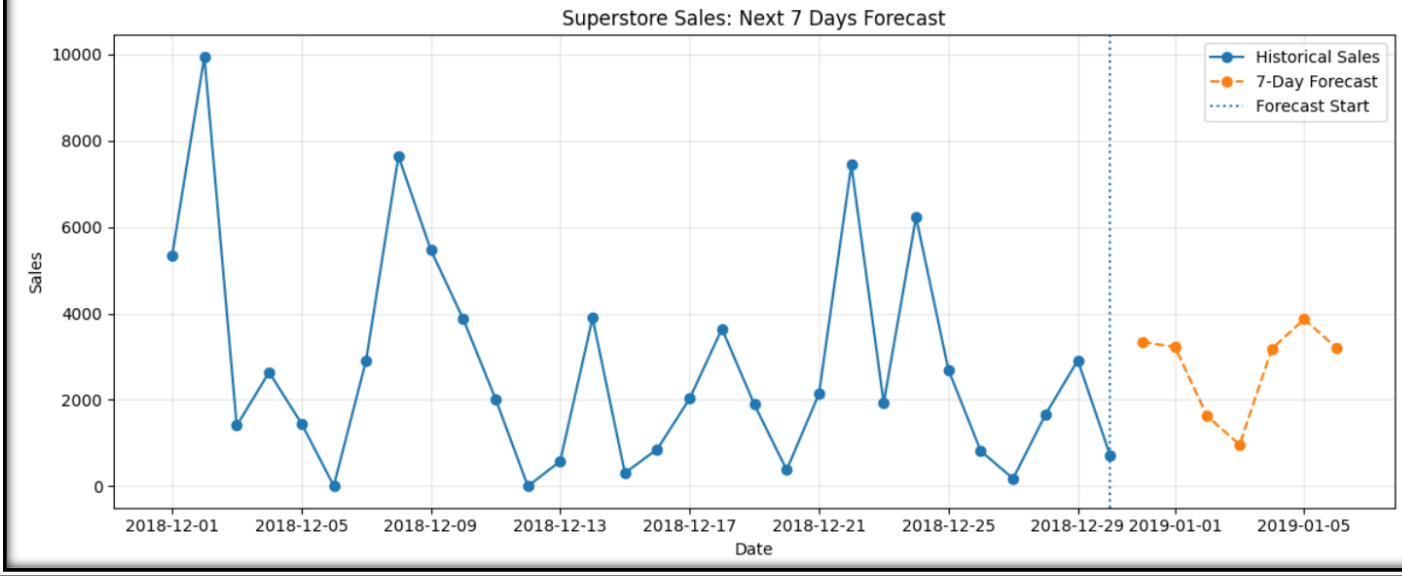
```
================ MODEL PERFORMANCE ================

              Model     MAE    RMSE
7-Day Moving Average 1632.15 2003.91
        Holt-Winters 1666.60 1833.09
Best Forecasting Model: Holt-Winters
```




```
================ NEXT 7 DAYS FORECAST ================

      Date  Forecasted Sales
2018-12-31           3338.29
2019-01-01           3226.96
2019-01-02           1623.37
2019-01-03            963.17
2019-01-04           3184.18
2019-01-05           3867.98
2019-01-06           3203.82
```



Learning Outcome:

PRACTICAL NO. 6

Aim: Write a program to Investigate the impact of feature selection and engineering techniques on predictive modeling performance.

Theory:

Predictive Business Analytics (PBA): Predictive Business Analytics (PBA) uses historical data, statistical techniques, and machine learning algorithms to predict future outcomes and support better business decisions. It helps organizations identify patterns, estimate outcomes, and understand factors affecting business results. The quality of a predictive model depends not only on the algorithm but also on the features provided to it. Feature selection and feature engineering are therefore important techniques in predictive analytics. In this experiment, PBA is applied to a House Purchase Prediction dataset to predict whether a customer will purchase a house. Logistic Regression and Decision Tree models are trained using different feature sets. Their performance is then compared using Accuracy, Precision, Recall, and F1-Score to investigate the effect of feature selection and engineering.

Feature Selection: Feature selection is the process of selecting the most relevant features from an existing dataset while removing less useful variables. Reducing unnecessary features can simplify a model, decrease computational requirements, and potentially improve predictive performance. In this experiment, SelectKBest with ANOVA F-test (f_classif) is used for feature selection. From the original 16 features, the 14 most relevant features are selected based on their relationship with the target variable, Purchased. The selected features are then used to train Logistic Regression and Decision Tree models. Their performance is compared with models trained using all original features. This allows the experiment to investigate whether removing two less relevant features affects the ability of the models to correctly predict house purchase decisions.

Feature Engineering: Feature engineering involves creating new meaningful variables from existing data to provide additional information to a machine learning model. In this experiment, six engineered features are created from the house purchase attributes. These include Price Per Square Foot, Income Price Ratio, Loan Burden, Down Payment Ratio, Expense to Income, and Space Per Person. For example, Price Per Square Foot relates house price to house area, while Loan Burden relates an existing loan to annual income. These derived features represent relationships that may be more informative than individual variables alone. The engineered feature set is then used with Logistic Regression and Decision Tree models. Comparing its performance with the original and selected feature sets helps determine whether feature engineering improves prediction.

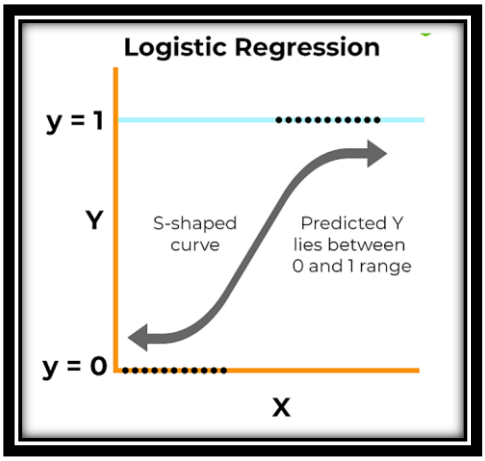
Classification Models Used: Two supervised classification algorithms are used in this experiment: Logistic Regression and Decision Tree. Logistic Regression is commonly used for binary classification and estimates the probability that an observation belongs to a

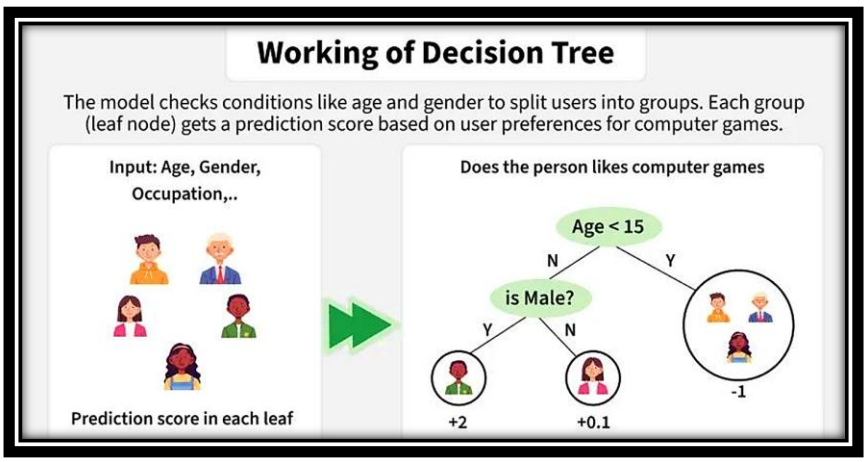
particular class. Since the target variable Purchased represents whether a customer purchases a house, Logistic Regression is suitable for this problem. Decision Tree classification uses a sequence of feature-based decisions to divide observations into groups and produce a final class prediction. It can capture nonlinear relationships and interactions between features. Logistic Regression requires feature scaling because the input variables have different numerical ranges, so StandardScaler is applied. Decision Trees do not require scaling. Both models are evaluated using the same feature sets for fair comparison.

Performance Evaluation and Comparison: The models are evaluated using Accuracy, Precision, Recall, and F1-Score. Accuracy represents the proportion of correctly classified observations out of all observations. Precision measures how many observations predicted as house purchases are actually purchases. Recall measures how many actual purchases are correctly identified by the model. F1-Score is the harmonic mean of Precision and Recall and provides a balanced measure of classification performance. These metrics are calculated for both Logistic Regression and Decision Tree using Original, Selected, and Engineered feature sets. Comparing these results helps determine whether feature selection or feature engineering improves predictive performance and identifies which combination of model and feature set produces the strongest results.

Dataset Description:

The House Purchase Prediction dataset contains information about 5,000 customers and includes 16 predictor features related to personal, financial, employment, and property characteristics. The features include Age, Annual Income, Credit Score, Family Size, Existing Loan, House Price, House Area, Bedrooms, Bathrooms, Distance to Work, Down Payment, Location Score, Employment Years, Monthly Expenses, Property Tax, and Maintenance Cost. The target variable is Purchased, which indicates whether a customer purchased a house or not. Therefore, the dataset represents a binary classification problem. In this experiment, the original 16 features are first used to establish a baseline. Feature selection is then performed using SelectKBest with the ANOVA F-test to retain 14 relevant features. Finally, six additional features are engineered from the original variables: Price Per Square Foot, Income Price Ratio, Loan Burden, Down Payment Ratio, Expense to Income, and Space Per Person. Logistic Regression and Decision Tree models are evaluated across all three feature sets.





Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score)

df = pd.read_csv("/content/house_purchase_prediction_features.csv")
X = df.drop("Purchased", axis=1)
y = df["Purchased"]
print("Dataset Shape     :", df.shape)
print("Number of Rows    :", df.shape[0])
print("Original Features :", X.shape[1])
print("Target Variable   : Purchased")

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y)

print("\nORIGINAL FEATURES\n")
for i, feature in enumerate(X.columns, 1):
    print(i, ".", feature)

selector = SelectKBest(score_func=f_classif,k=14)

X_train_selected = selector.fit_transform(
    X_train,
    y_train)
X_test_selected = selector.transform(
    X_test)
selected_features = list(
    X_train.columns[selector.get_support()])
```

```
print("\nSELECTED FEATURES\n")

for i, feature in enumerate(selected_features, 1):
    print(i, ".", feature)

X_engineered = X.copy()
X_engineered["Price_Per_SqFt"] = (
    X_engineered["House_Price"] /
    X_engineered["House_Area"])

X_engineered["Income_Price_Ratio"] = (
    X_engineered["Annual_Income"] /
    X_engineered["House_Price"])

X_engineered["Loan_Burden"] = (
    X_engineered["Existing_Loan"] /
    X_engineered["Annual_Income"])

X_engineered["DownPayment_Ratio"] = (
    X_engineered["Down_Payment"] /
    X_engineered["House_Price"])

X_engineered["Expense_to_Income"] = (
    X_engineered["Monthly_Expenses"] /
    (X_engineered["Annual_Income"] / 12))

X_engineered["Space_Per_Person"] = (
    X_engineered["House_Area"] /
    X_engineered["Family_Size"])

X_engineered = X_engineered.replace(
    [np.inf, -np.inf],
    np.nan).fillna(0)

engineered_features = [
    "Price_Per_SqFt",
    "Income_Price_Ratio",
    "Loan_Burden",
    "DownPayment_Ratio",
    "Expense_to_Income",
    "Space_Per_Person"]
```

```
print("\nENGINEERED FEATURES\n")

for i, feature in enumerate(engineered_features, 1):
    print(i, ".", feature)

print("\nFEATURE COUNT\n")
print("Original   :", X.shape[1])
print("Selected   :", len(selected_features))
print("Engineered :", X_engineered.shape[1])

X_train_eng, X_test_eng, y_train_eng, y_test_eng = train_test_split(
    X_engineered,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y)

def evaluate(model, Xtr, Xte, ytr, yte):
    model.fit(Xtr, ytr)
    prediction = model.predict(Xte)

    return [
        accuracy_score(yte, prediction),
        precision_score(yte, prediction, zero_division=0),
        recall_score(yte, prediction, zero_division=0),
        f1_score(yte, prediction, zero_division=0)]

results = []

scaler = StandardScaler()

Xtr = scaler.fit_transform(X_train)
Xte = scaler.transform(X_test)

log_original = LogisticRegression(
    max_iter=1000,
    random_state=42)

m = evaluate(
    log_original,
    Xtr,
    Xte,
    y_train,
    y_test)

results.append(
    ["Logistic Regression", "Original"] + m)
```

```
scaler = StandardScaler()
Xtr = scaler.fit_transform(X_train_selected)
Xte = scaler.transform(X_test_selected)
log_selected = LogisticRegression(
    max_iter=1000,
    random_state=42)

m = evaluate(
    log_selected,
    Xtr,
    Xte,
    y_train,
    y_test)

results.append(
    ["Logistic Regression", "Selected"] + m)

scaler = StandardScaler()
Xtr = scaler.fit_transform(X_train_eng)
Xte = scaler.transform(X_test_eng)
log_engineered = LogisticRegression(
    max_iter=1000,
    random_state=42)

m = evaluate(
    log_engineered,
    Xtr,
    Xte,
    y_train_eng,
    y_test_eng)
results.append(
    ["Logistic Regression", "Engineered"] + m)

tree_original = DecisionTreeClassifier(
    max_depth=6,
    random_state=42)

m = evaluate(
    tree_original,
    X_train,
    X_test,
    y_train,
    y_test)
```

```
results.append(
    ["Decision Tree", "Original"] + m)
tree_selected = DecisionTreeClassifier(
    max_depth=6,
    random_state=42)
m = evaluate(
    tree_selected,
    X_train[selected_features],
    X_test[selected_features],
    y_train,
    y_test)
results.append(
    ["Decision Tree", "Selected"] + m)
tree_engineered = DecisionTreeClassifier(
    max_depth=6,
    random_state=42)

m = evaluate(
    tree_engineered,
    X_train_eng,
    X_test_eng,
    y_train_eng,
    y_test_eng)

results.append(
    ["Decision Tree", "Engineered"] + m)
results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "Feature Set",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"])
print("\n\nFINAL RESULTS\n")

print(results_df.round(3).to_string(index=False))
metrics = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1 Score"]
```

```
log_results = results_df[
    results_df["Model"] == "Logistic Regression"]
x = np.arange(3)
width = 0.18
plt.figure(figsize=(9, 5))
for i, metric in enumerate(metrics):
    plt.bar(
        x + (i - 1.5) * width,
        log_results[metric],
        width,
        label=metric)
plt.xticks(x,["Original", "Selected", "Engineered"])
plt.xlabel("Feature Set")
plt.ylabel("Score")
plt.title("Logistic Regression Performance")
plt.ylim(0, 1)
plt.legend()
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

tree_results = results_df[
    results_df["Model"] == "Decision Tree"]
plt.figure(figsize=(9, 5))
for i, metric in enumerate(metrics):
    plt.bar(
        x + (i - 1.5) * width,
        tree_results[metric],
        width,
        label=metric)
plt.xticks(x,["Original", "Selected", "Engineered"])
plt.xlabel("Feature Set")
plt.ylabel("Score")
plt.title("Decision Tree Performance")
plt.ylim(0, 1)
plt.legend()
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

best = results_df.loc[
    results_df["F1 Score"].idxmax()]

print("\nBEST RESULT")
print("Model       :", best["Model"])
print("Feature Set :", best["Feature Set"])
print("Accuracy    :", round(best["Accuracy"], 3))
print("Precision   :", round(best["Precision"], 3))
print("Recall      :", round(best["Recall"], 3))
print("F1 Score    :", round(best["F1 Score"], 3))
```

Output:

```
Dataset Shape      : (5000, 17)
Number of Rows     : 5000
Original Features : 16
Target Variable    : Purchased

ORIGINAL FEATURES

1 . Age
2 . Annual_Income
3 . Credit_Score
4 . Family_Size
5 . Existing_Loan
6 . House_Price
7 . House_Area
8 . Bedrooms
9 . Bathrooms
10 . Distance_to_Work
11 . Down_Payment
12 . Location_Score
13 . Employment_Years
14 . Monthly_Expenses
15 . Property_Tax
16 . Maintenance_Cost

SELECTED FEATURES

1 . Age
2 . Annual_Income
3 . Credit_Score
4 . Family_Size
5 . Existing_Loan
6 . House_Price
7 . House_Area
8 . Bedrooms
9 . Bathrooms
10 . Down_Payment
11 . Location_Score
12 . Monthly_Expenses
13 . Property_Tax
14 . Maintenance_Cost

ENGINEERED FEATURES

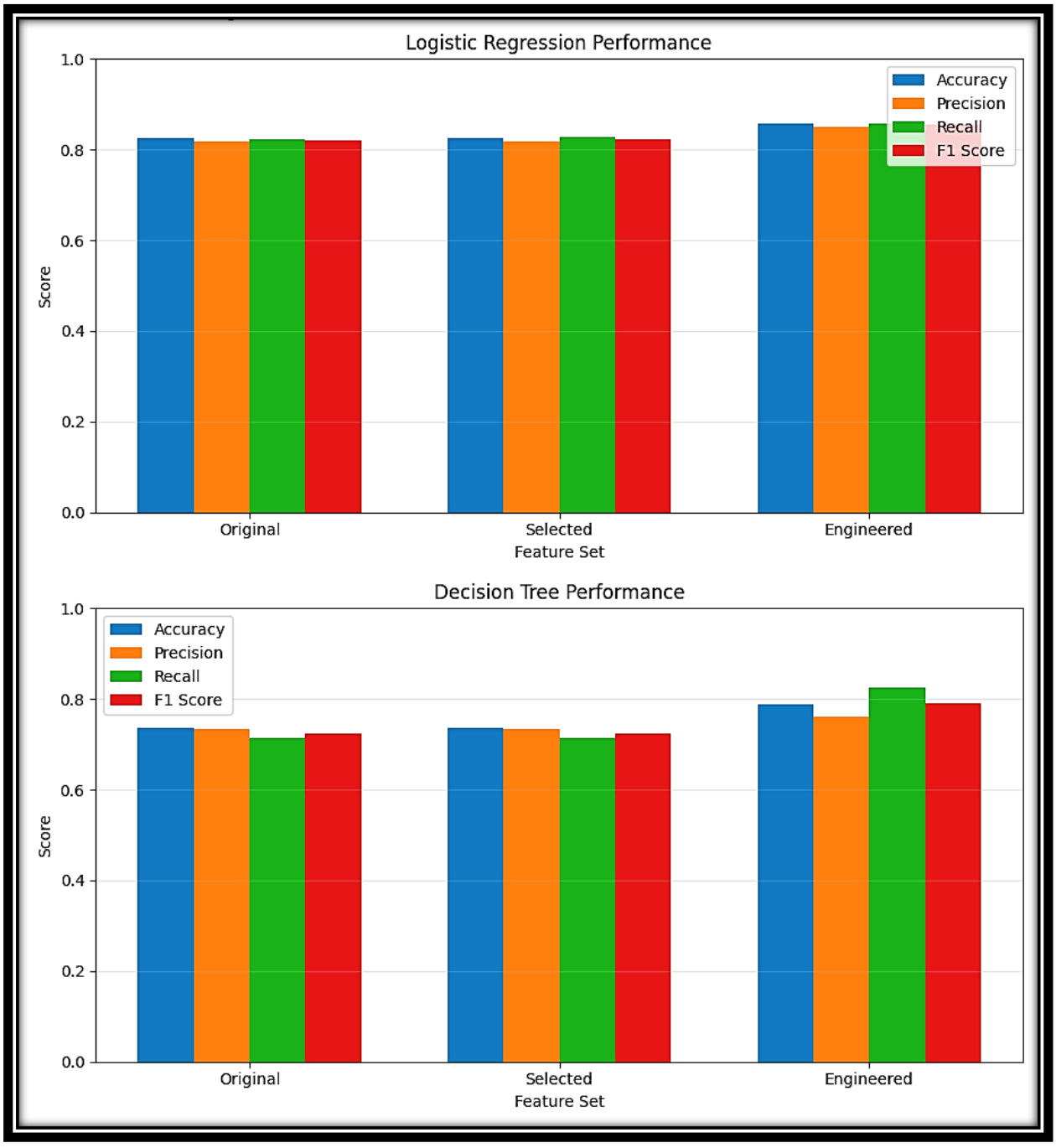
1 . Price_Per_SqFt
2 . Income_Price_Ratio
3 . Loan_Burden
4 . DownPayment_Ratio
5 . Expense_to_Income
6 . Space_Per_Person

FEATURE COUNT

Original    : 16
Selected    : 14
Engineered : 22
```

FINAL RESULTS

Model	Feature Set	Accuracy	Precision	Recall	F1 Score
Logistic Regression	Original	0.825	0.818	0.823	0.820
Logistic Regression	Selected	0.826	0.817	0.827	0.822
Logistic Regression	Engineered	0.858	0.851	0.858	0.854
Decision Tree	Original	0.736	0.735	0.713	0.724
Decision Tree	Selected	0.736	0.735	0.713	0.724
Decision Tree	Engineered	0.789	0.760	0.825	0.791



Learning Outcome:

PRACTICAL NO. 7

Aim: Write a program to experiment with different methods such as correlation analysis, stepwise regression, or Recursive Feature Elimination (RFE) to identify the most relevant features. Compare the performance of models with and without feature engineering.

Theory:

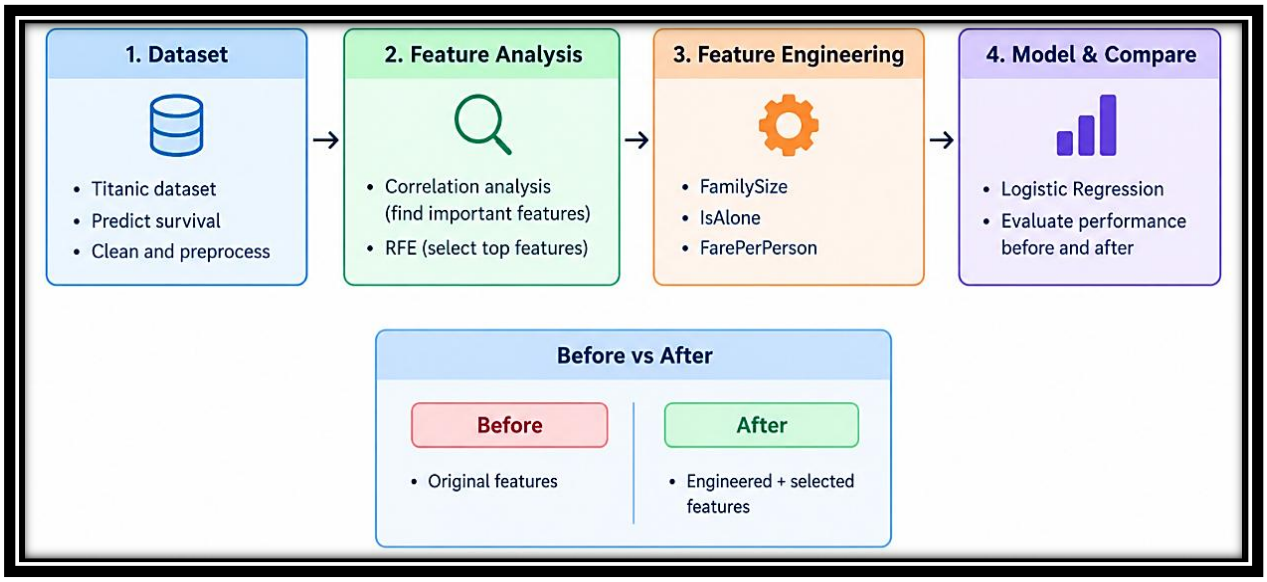
Predictive Business Analytics (PBA): It uses historical data, statistical techniques, and machine learning algorithms to identify patterns and predict future outcomes. An important part of predictive modelling is selecting useful features because irrelevant or redundant variables may affect model performance. Feature selection methods help identify the variables that provide useful information for prediction, while feature engineering creates new variables from existing data. In this experiment, PBA concepts are applied to the Titanic dataset to predict passenger survival. Correlation analysis and Recursive Feature Elimination (RFE) are used to study and select relevant features. Feature engineering is also performed along with Logistic Regression to create additional passenger-related variables and compare model performance before and after feature engineering and RFE.

Correlation Analysis: It is a statistical technique used to measure the strength and direction of the relationship between numerical variables. The correlation coefficient generally ranges from -1 to +1, where values closer to +1 indicate a positive relationship, values closer to -1 indicate a negative relationship and values near zero indicate a weak linear relationship. In this experiment, correlation analysis is performed to examine the relationship between passenger features and the Survived target variable. The absolute correlation values are displayed and visualized using a bar graph to identify features with stronger relationships with survival. RFE is additionally used for more systematic feature selection.

Recursive Feature Elimination (RFE): It is a feature selection technique that identifies important features by repeatedly eliminating less useful variables. It begins with all available features, trains an estimator, evaluates the importance of the features, and removes the least important ones. This process continues until the specified number of features remains. In this experiment, Logistic Regression is used as the estimator for RFE, and the six most relevant features are selected from the pre-processed and engineered Titanic dataset. The selected features are then used to train the final Logistic Regression model. RFE helps reduce the number of input variables while retaining features that contribute more strongly to prediction. Its results are compared with the baseline model that uses the original features.

Feature Engineering: Feature engineering involves creating new meaningful variables from existing features to provide additional information to a machine learning model. In this experiment, three new features are created from the Titanic passenger information. FamilySize is calculated using the number of siblings or spouses and parents or children travelling with the passenger. IsAlone indicates whether a passenger travelled alone, while

FarePerPerson represents the fare relative to the passenger's family size. These features can represent passenger and travel characteristics more clearly than the original variables alone. After creating the engineered features, RFE is applied to identify the most relevant variables. The resulting feature set is then used to train the final Logistic Regression model and determine whether feature engineering improves predictive performance.



Model Evaluation and Comparison: Logistic Regression is used as the classification model because the Survived variable represents a binary outcome. Before feature engineering, the model is trained using the original preprocessed features and evaluated using Accuracy, Precision, Recall, and F1-Score. After feature engineering, RFE selects six relevant features, and another Logistic Regression model is trained using these selected features. The two models are compared using the same evaluation metrics. Accuracy measures the proportion of correct predictions, Precision measures the correctness of positive predictions, and Recall measures how many actual survivors are correctly identified. F1-Score combines Precision and Recall into a balanced measure. In this experiment, the baseline F1-score is 0.743, while the final model achieves 0.777, showing an improvement after feature engineering and RFE.

Dataset Description:

The Titanic dataset contains information about passengers who travelled on the RMS Titanic and records whether each passenger survived. The dataset used in this experiment is obtained from a public Titanic CSV dataset and contains passenger information such as passenger class, sex, age, number of siblings or spouses, number of parents or children, fare, and port of embarkation. The target variable is Survived, which indicates whether a passenger survived, making the problem a binary classification task. Before modeling, rows containing missing values are removed. The Sex feature is converted into numerical form, with male represented as 0 and female as 1, while the Embarked feature is converted using one-hot encoding. Three additional engineered features are created: FamilySize, IsAlone, and FarePerPerson. Correlation analysis is used to examine relationships with survival, and RFE selects six relevant features. Logistic Regression is then used to compare performance before and after feature engineering and RFE.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score)

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)
print("Dataset Shape:", df.shape)

df = df[
    [   "Survived",
        "Pclass",
        "Sex",
        "Age",
        "SibSp",
        "Parch",
        "Fare",
        "Embarked"]]
df = df.dropna().copy()

df["Sex"] = df["Sex"].map({
    "male": 0,
    "female": 1})

df = pd.get_dummies(
    df,
    columns=["Embarked"],
    drop_first=True,
    dtype=int)
```

```
print("\nData After Preprocessing:\n")
print(df.head())

X = df.drop("Survived", axis=1)
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
baseline_model = LogisticRegression(
    max_iter=1000,
    random_state=42)
baseline_model.fit(
    X_train_scaled,
    y_train)
y_pred = baseline_model.predict(
    X_test_scaled)
accuracy_before = accuracy_score(
    y_test,
    y_pred)
precision_before = precision_score(
    y_test,
    y_pred,
    zero_division=0)
recall_before = recall_score(
    y_test,
    y_pred,
    zero_division=0)
```

```
f1_before = f1_score(
    y_test,
    y_pred,
    zero_division=0)

df["FamilySize"] = (
    df["SibSp"] +
    df["Parch"] + 1)

df["IsAlone"] = (
    df["FamilySize"] == 1).astype(int)

df["FarePerPerson"] = (
    df["Fare"] /
    df["FamilySize"])

print("\nENGINEERED FEATURES:\n")
print(
    df[["FamilySize",
        "IsAlone",
        "FarePerPerson"]].head())
X_new = df.drop("Survived", axis=1)
y_new = df["Survived"]
X_train_new, X_test_new, y_train_new, y_test_new = train_test_split(
    X_new,
    y_new,
    test_size=0.20,
    random_state=42,
    stratify=y_new)
correlation = (
    df.corr(numeric_only=True)["Survived"]
    .drop("Survived")
    .sort_values(
        ascending=False))

print("\nCORRELATION WITH SURVIVAL:\n")
```

```
print(correlation.round(3))
plt.figure(figsize=(9, 5))
correlation.abs().sort_values(ascending=False).plot(kind="bar")
plt.title("Feature Correlation with Survival")
plt.xlabel("Features")
plt.ylabel("Absolute Correlation")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

scaler_new = StandardScaler()
X_train_new_scaled = scaler_new.fit_transform(X_train_new)
X_test_new_scaled = scaler_new.transform(X_test_new)
rfe_model = LogisticRegression(max_iter=1000, random_state=42)

rfe = RFE(estimator=rfe_model, n_features_to_select=6)
rfe.fit(X_train_new_scaled,y_train_new)
selected_features = X_new.columns[rfe.support_]
print("\nFEATURES SELECTED BY RFE:\n")
for i, feature in enumerate(selected_features, 1):print(i, ".", feature)

X_train_selected = (X_train_new_scaled[:, rfe.support_])
X_test_selected = (X_test_new_scaled[:, rfe.support_])
final_model = LogisticRegression(max_iter=1000, random_state=42)
final_model.fit(X_train_selected,y_train_new)

y_pred_final = final_model.predict(X_test_selected)
accuracy_after = accuracy_score(y_test_new, y_pred_final)
precision_after = precision_score(y_test_new, y_pred_final, zero_division=0)
recall_after = recall_score(y_test_new, y_pred_final, zero_division=0)
f1_after = f1_score(y_test_new, y_pred_final, zero_division=0)

results = pd.DataFrame({
        "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"],
    "Before Feature Engineering": [
        accuracy_before,
        precision_before,
        recall_before,
        f1_before],
    "After Feature Engineering + RFE": [
        accuracy_after,
        precision_after,
        recall_after,
        f1_after]})
```

```
print("\n================ COMPARISON TABLE ================\n")
print(results.round(3).to_string(index=False))
results_plot = results.set_index("Metric")
results_plot.plot(kind="bar", figsize=(9, 5))

plt.title("Model Performance Before and After Feature Engineering")
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend()
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

print("Baseline F1 Score :", round(f1_before, 3))
print("Final F1 Score    :", round(f1_after, 3))
```

Output:

```
Dataset Shape: (891, 12)

Data After Preprocessing:

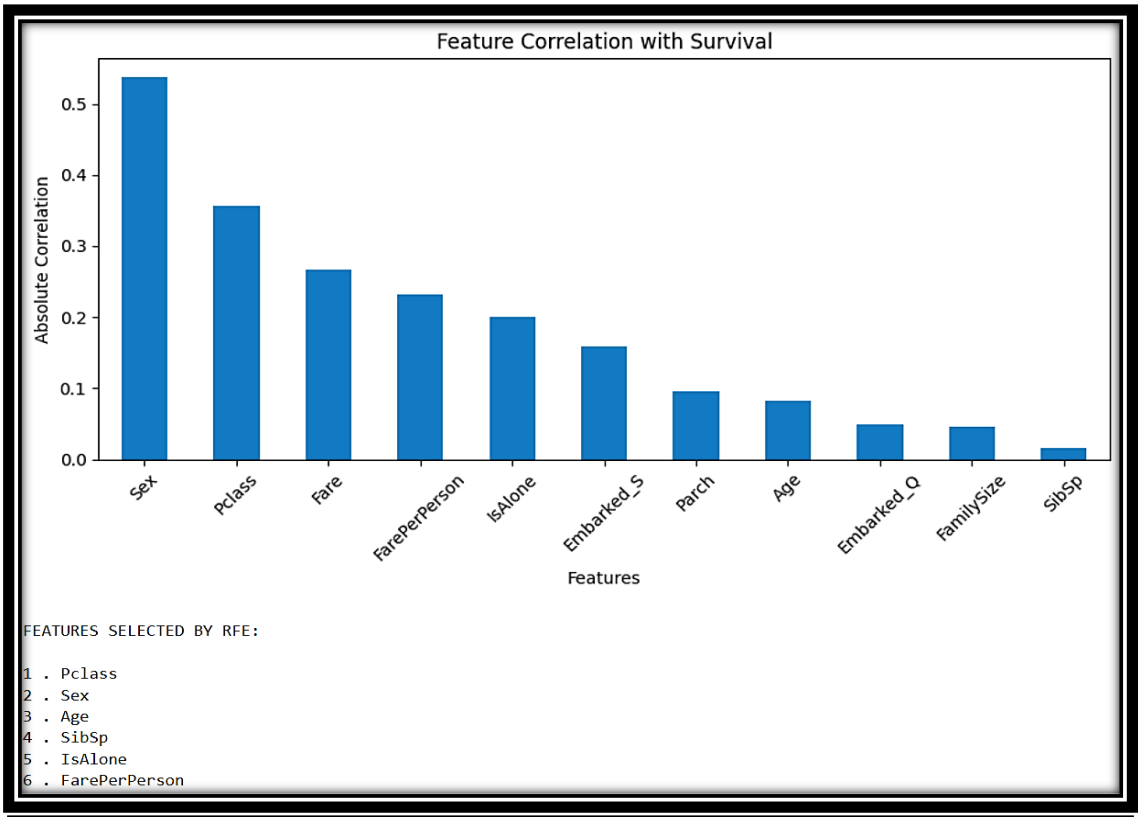
   Survived  Pclass  Sex   Age  SibSp  Parch     Fare  Embarked_Q  Embarked_S
0         0       3    0  22.0      1      0   7.2500           0           1
1         1       1    1  38.0      1      0  71.2833           0           0
2         1       3    1  26.0      0      0   7.9250           0           1
3         1       1    1  35.0      1      0  53.1000           0           1
4         0       3    0  35.0      0      0   8.0500           0           1

ENGINEERED FEATURES:

   FamilySize  IsAlone  FarePerPerson
0           2        0        3.62500
1           2        0       35.64165
2           1        1        7.92500
3           2        0       26.55000
4           1        1        8.05000

CORRELATION WITH SURVIVAL:

Sex              0.537
Fare             0.266
FarePerPerson    0.232
Parch            0.095
FamilySize       0.045
SibSp           -0.016
Embarked_Q      -0.049
Age             -0.082
Embarked_S      -0.159
IsAlone         -0.200
Pclass          -0.356
Name: Survived, dtype: float64
```

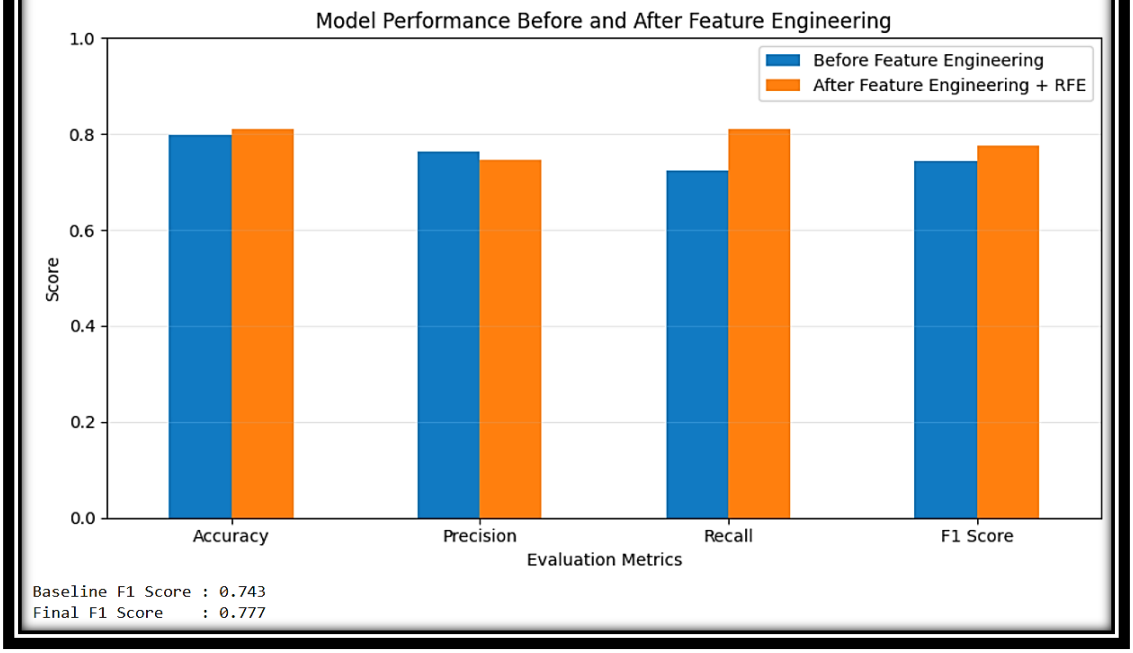



```
FEATURES SELECTED BY RFE:

1 . Pclass
2 . Sex
3 . Age
4 . SibSp
5 . IsAlone
6 . FarePerPerson
```

```
================ COMPARISON TABLE ================

   Metric  Before Feature Engineering  After Feature Engineering + RFE
 Accuracy                       0.797                           0.811
Precision                       0.764                           0.746
   Recall                       0.724                           0.810
 F1 Score                       0.743                           0.777
```




```
Baseline F1 Score : 0.743
Final F1 Score    : 0.777
```

Learning Outcome:

PRACTICAL NO. 8

Aim: Write a program to explore the effectiveness of ensemble methods, such as bagging, boosting, or stacking, in improving predictive performance.

Theory:

Predictive Business Analytics (PBA): - Predictive Business Analytics (PBA) uses historical data, statistical methods, and machine learning algorithms to predict outcomes and support decision-making. Businesses apply predictive analytics to understand customer behaviour, estimate demand, identify risks, and improve operational efficiency. Machine learning models learn patterns from existing data and use them to make predictions on unseen observations. However, a single model may not always provide the most accurate or reliable results. Ensemble learning addresses this limitation by combining multiple models to improve predictive performance. In this experiment, PBA concepts are applied to the Titanic dataset to predict passenger survival. Individual classification models are compared with ensemble methods using standard evaluation metrics to investigate whether combining models improves the accuracy and reliability of survival predictions.

Ensemble Learning: - Ensemble learning is a machine learning technique that combines predictions from multiple models to produce a final prediction. Instead of relying on a single classifier, ensemble methods use several models to improve predictive performance, reduce errors, or increase stability. Different ensemble techniques combine models in different ways. Bagging trains multiple models on different samples of the training data and combines their predictions. Boosting trains models sequentially, allowing later models to focus on errors made by earlier ones. Stacking combines predictions from different base models using another model called a meta-model. In this experiment, Bagging, Random Forest, AdaBoost, Gradient Boosting, and Stacking are investigated to understand their effectiveness for predicting Titanic passenger survival.

Bagging and Random Forest: - Bagging, or Bootstrap Aggregating, is an ensemble technique that trains multiple models on randomly sampled versions of the training dataset. Their predictions are combined to produce a final result, which can reduce variance and help prevent overfitting. Random Forest is a popular bagging-based method that constructs multiple decision trees using random samples and subsets of features. For classification, the trees' predictions are combined through voting. These techniques are useful because individual decision trees can be sensitive to changes in training data. In this experiment, Bagging and Random Forest classifiers are trained on the Titanic passenger features. Their performance is compared with individual Logistic Regression and Decision Tree models to determine whether combining multiple trees improves survival prediction.

Boosting and Stacking: - Boosting builds an ensemble by combining models sequentially, with later models attempting to improve on earlier predictions. AdaBoost adjusts the

importance of training observations so that misclassified cases receive greater attention, while Gradient Boosting builds new models to reduce the remaining prediction errors. Stacking uses a different approach: predictions from multiple base models become inputs to a meta-model, which learns how to combine them. In this experiment, AdaBoost, Gradient Boosting, and Stacking are used alongside bagging-based methods. The Titanic dataset provides a binary classification problem in which the target variable indicates survival or non-survival. Comparing these techniques helps demonstrate how different ensemble strategies can capture patterns and improve predictions compared with using a single model.

Model Evaluation and Comparison: - The performance of the classification models is evaluated using Accuracy, Precision, Recall, and F1-Score. Accuracy measures the proportion of passengers classified correctly. Precision indicates how many passengers predicted as survivors actually survived, while Recall measures how many actual survivors were correctly identified. F1-Score is the harmonic mean of Precision and Recall, providing a balanced measure of both. In this experiment, Logistic Regression and Decision Tree serve as individual-model baselines, while Bagging, Random Forest, AdaBoost, Gradient Boosting, and Stacking represent ensemble approaches. All models use the same training and testing split to support a fair comparison. The evaluation results and performance graph are used to identify the best-performing approach based on its F1-Score.

Dataset Description: The Titanic dataset contains passenger information from the RMS Titanic and records whether each passenger survived the disaster. In this experiment, the dataset is obtained from a publicly available CSV file and includes variables such as passenger class (Pclass), sex (Sex), age (Age), number of siblings or spouses (SibSp), number of parents or children (Parch), ticket fare (Fare), and port of embarkation (Embarked). The target variable is Survived, where 0 represents non-survival and 1 represents survival, making this a binary classification problem. Rows containing missing values in the selected fields are removed, and categorical variables are converted into numerical representations. Three additional features—FamilySize, IsAlone, and FarePerPerson—are created through feature engineering. The dataset is divided into training and testing sets. Individual classifiers and ensemble methods are trained on the same data split and evaluated using Accuracy, Precision, Recall, and F1-Score to determine which approach predicts passenger survival most effectively.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    BaggingClassifier,
    RandomForestClassifier,
    AdaBoostClassifier,
    GradientBoostingClassifier,
    StackingClassifier)
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score)

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)
df = df[
    ["Survived", "Pclass", "Sex", "Age", "SibSp",
     "Parch", "Fare", "Embarked"]].dropna().copy()

df["Sex"] = df["Sex"].map({"male": 0, "female": 1})
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
df["FarePerPerson"] = df["Fare"] / df["FamilySize"]

df = pd.get_dummies(
    df, columns=["Embarked"], drop_first=True, dtype=int)

print("Dataset Shape:", df.shape)
print("Number of Rows:", df.shape[0])
print("Target Variable: Survived")

X = df.drop("Survived", axis=1)
y = df["Survived"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

models = {
    "Logistic Regression": LogisticRegression(
        max_iter=1000, random_state=42
    ),
    "Decision Tree": DecisionTreeClassifier(
        max_depth=5, random_state=42
    ),
    "Bagging": BaggingClassifier(
        estimator=DecisionTreeClassifier(random_state=42),
        n_estimators=100, random_state=42, n_jobs=-1
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, max_depth=5,
        random_state=42, n_jobs=-1
    ),
    "AdaBoost": AdaBoostClassifier(
        n_estimators=100, random_state=42
    ),
    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=100, max_depth=2, random_state=42
    ),
    "Stacking": StackingClassifier(
        estimators=[
            ("lr", LogisticRegression(max_iter=1000)),
            ("dt", DecisionTreeClassifier(max_depth=5, random_state=42)),
            ("rf", RandomForestClassifier(
                n_estimators=50, max_depth=5, random_state=42, n_jobs=-1))],
        final_estimator=LogisticRegression(max_iter=1000),
        cv=5)}
```

```
results = []

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    prediction = model.predict(X_test_scaled)

    results.append([
        name,
        accuracy_score(y_test, prediction),
        precision_score(y_test, prediction, zero_division=0),
        recall_score(y_test, prediction, zero_division=0),
        f1_score(y_test, prediction, zero_division=0)])

results_df = pd.DataFrame(
    results,
    columns=["Model", "Accuracy", "Precision", "Recall", "F1-Score"])

print("\nMODEL PERFORMANCE COMPARISON\n")
print(results_df.round(3).to_string(index=False))

best = results_df.loc[results_df["F1-Score"].idxmax()]

print("\nBEST MODEL")
print("Model     :", best["Model"])
print("Accuracy  :", round(best["Accuracy"], 3))
print("Precision :", round(best["Precision"], 3))
print("Recall    :", round(best["Recall"], 3))
print("F1-Score  :", round(best["F1-Score"], 3))

results_df.set_index("Model")[
    ["Accuracy", "Precision", "Recall", "F1-Score"]].plot(kind="bar", figsize=(12, 6))

plt.title("Individual Models vs Ensemble Methods")
plt.xlabel("Classification Model")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=35, ha="right")
plt.legend(loc="lower right")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```

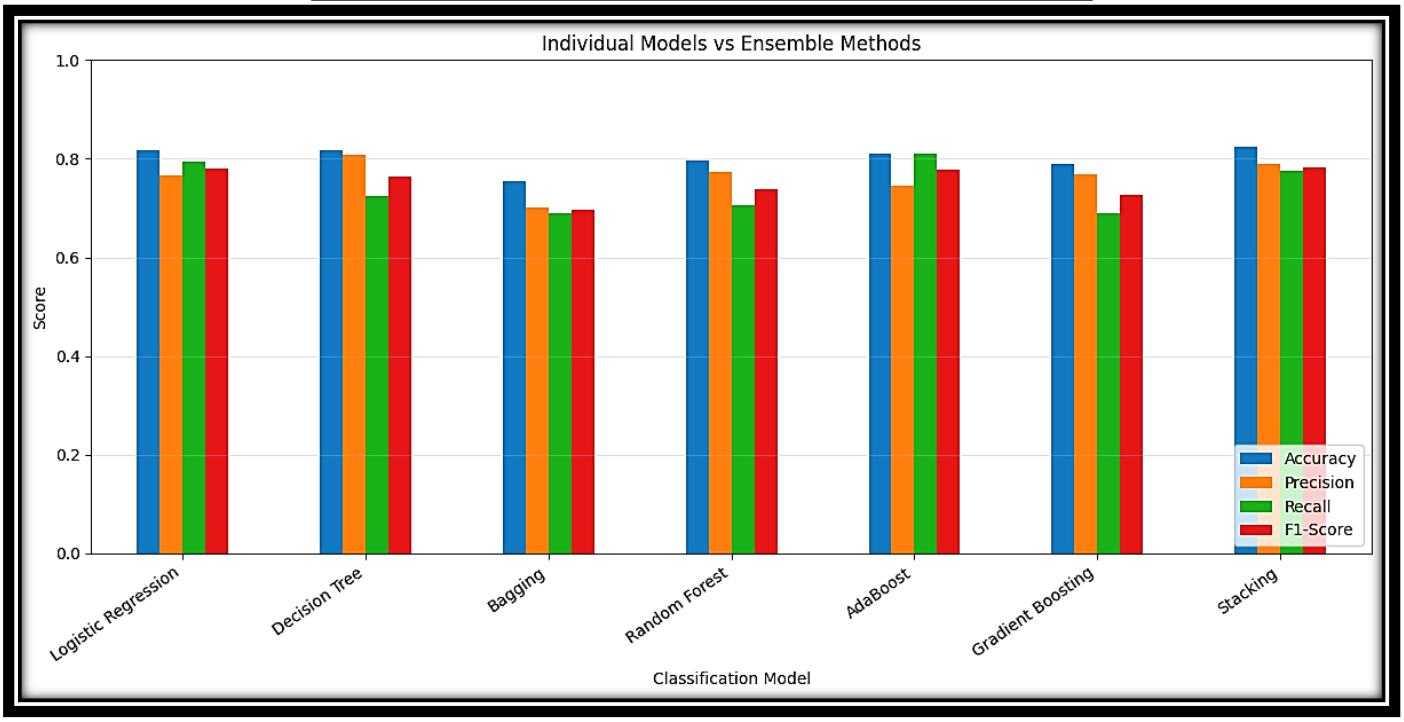
Output:

```
Dataset Shape: (712, 12)
Number of Rows: 712
Target Variable: Survived

MODEL PERFORMANCE COMPARISON

              Model  Accuracy  Precision  Recall  F1-Score
Logistic Regression     0.818      0.767   0.793     0.780
      Decision Tree     0.818      0.808   0.724     0.764
            Bagging     0.755      0.702   0.690     0.696
      Random Forest     0.797      0.774   0.707     0.739
           AdaBoost     0.811      0.746   0.810     0.777
  Gradient Boosting     0.790      0.769   0.690     0.727
           Stacking     0.825      0.789   0.776     0.783

BEST MODEL
Model     : Stacking
Accuracy  : 0.825
Precision : 0.789
Recall    : 0.776
F1-Score  : 0.783
```



Learning Outcome:

PRACTICAL NO. 9

Aim: Write a program to analyse textual data, such as customer reviews or social media comments, to extract sentiment and gain insights into customer opinions. Apply techniques like Natural Language Processing (NLP), text preprocessing, and sentiment analysis algorithms to classify text into positive, negative, or neutral sentiments.

Theory:

Introduction to NLP: Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, process, and analyse human language. Businesses receive large amounts of textual data through customer reviews, feedback forms, social media comments, and online discussions. Manually analysing this information can be time-consuming. NLP techniques help process text automatically and identify useful patterns in customer opinions. Sentiment analysis is an important NLP application that determines whether a piece of text expresses a positive, negative, or neutral opinion. It helps organisations understand customer satisfaction, identify complaints, evaluate products, and make data-driven business decisions.

Text preprocessing: Text preprocessing converts raw text into a suitable format for machine learning. Social media text may contain punctuation, URLs, repeated characters, spelling variations, and unnecessary symbols. Common preprocessing techniques include converting text to lowercase, removing unwanted elements, handling missing values, and tokenisation. Stop-word removal and stemming or lemmatisation may also be used when appropriate. However, words such as not and never can change the meaning of a sentence, so removing them carelessly may reduce sentiment classification performance. Proper preprocessing helps reduce noise and allows the model to focus on meaningful words and expressions.

Feature extraction using TF-IDF: Machine learning algorithms cannot directly understand text, so textual information must be converted into numerical features. Term Frequency–Inverse Document Frequency (TF-IDF) represents words according to their importance within a document relative to the entire collection. Common words receive lower importance when they occur across many documents, while more distinctive terms can receive higher weights. Word n-grams capture individual words and short phrases, whereas character n-grams can capture spelling variations and informal writing patterns. Combining these features is useful for social media data, where users frequently use abbreviations, misspellings, and unconventional expressions.

Sentiment classification using Linear SVM: Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification. A Linear SVM attempts to find decision boundaries that separate different sentiment classes in the feature space. It is well suited to text classification because TF-IDF representations are typically high-

dimensional and sparse. In this experiment, the model predicts one of three classes: negative, neutral, or positive. The model learns from labelled examples in the training dataset and predicts sentiments for unseen test data. Using separate training and testing datasets helps evaluate whether the model can generalise beyond the examples it has already seen.

Evaluation and business applications: The performance of a sentiment analysis model can be evaluated using accuracy, precision, recall, and F1-score. Accuracy measures the proportion of correctly classified examples, while precision measures how many predictions for a class are correct. Recall measures how many actual examples of a class are identified, and the F1-score balances precision and recall. A confusion matrix can reveal which sentiment classes are commonly misclassified. Businesses can use these results to monitor customer satisfaction, identify recurring complaints, compare public reactions to products, and track changes in customer opinions. Sentiment analysis supports decision-making, although sarcasm and ambiguous language can still make automatic predictions difficult.

Dataset Description:

This experiment uses the TweetEval Sentiment dataset, a collection of social media text labelled for sentiment classification. Each record contains a tweet and a sentiment label. The dataset supports three categories: negative, neutral, and positive. It provides predefined training, validation, and testing splits, allowing the model to learn from labelled training examples and be evaluated on separate, unseen examples. Social media text is useful for this experiment because it contains informal language, short phrases, abbreviations, and varied expressions of opinion. The dataset is suitable for studying text preprocessing, TF-IDF feature extraction, supervised classification, and sentiment evaluation. The experiment uses word-level and character-level features with a Linear SVM classifier and evaluates its predictions using standard classification metrics.

Source Code:

```
%pip -q install scikit-learn requests matplotlib

import re
import requests
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import FeatureUnion
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, ConfusionMatrixDisplay

base_url = "https://raw.githubusercontent.com/cardiffnlp/tweeteval/main/datasets/sentiment/"

def download_file(filename):
    response = requests.get(base_url + filename, timeout=60)
    response.raise_for_status()
    return response.text.splitlines()

train_text = download_file("train_text.txt")
train_y = np.array(download_file("train_labels.txt"), dtype=int)
test_text = download_file("test_text.txt")
test_y = np.array(download_file("test_labels.txt"), dtype=int)
```

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r"https?://\S+|www\.\S+", " URL ", text)
    text = re.sub(r"@\w+", " USER ", text)
    text = re.sub(r"#(\w+)", r"\1", text)
    text = re.sub(r"(.)\1{2,}", r"\1\1", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

train_text = [preprocess(t) for t in train_text]
test_text = [preprocess(t) for t in test_text]

vectorizer = FeatureUnion([
    ("word", TfidfVectorizer(
        ngram_range=(1, 2),
        min_df=2,
        max_features=150000,
        sublinear_tf=True,
        strip_accents="unicode")),
    ("char", TfidfVectorizer(
        analyzer="char",
        ngram_range=(3, 5),
        min_df=2,
        max_features=100000,
        sublinear_tf=True,
        max_df=0.98))])

X_train = vectorizer.fit_transform(train_text)
X_test = vectorizer.transform(test_text)

model = LinearSVC(C=1.0, max_iter=5000)
model.fit(X_train, train_y)
predictions = model.predict(X_test)
```

```
print("NLP SENTIMENT ANALYSIS")
print("-" * 40)
print("Training samples:", len(train_text))
print("Testing samples:", len(test_text))
print("Accuracy:", round(accuracy_score(test_y, predictions) * 100, 2), "%")
print("Precision:", round(precision_score(test_y, predictions, average="weighted", zero_division=0), 4))
print("Recall:", round(recall_score(test_y, predictions, average="weighted", zero_division=0), 4))
print("F1 Score:", round(f1_score(test_y, predictions, average="weighted", zero_division=0), 4))

print("\nClassification Report:\n")
print(classification_report(
    test_y,
    predictions,
    labels=[0, 1, 2],
    target_names=["Negative", "Neutral", "Positive"],
    zero_division=0))

ConfusionMatrixDisplay.from_predictions(
    test_y,
    predictions,
    labels=[0, 1, 2],
    display_labels=["Negative", "Neutral", "Positive"],
    cmap="Blues")
plt.title("Sentiment Analysis Confusion Matrix")
plt.tight_layout()
plt.show()

examples = [
    "I absolutely love this product, it is amazing!",
    "This is the worst experience I have ever had.",
    "The product arrived today and works as expected."]

print("\nSAMPLE PREDICTIONS")
for text in examples:
    features = vectorizer.transform([preprocess(text)])
    prediction = model.predict(features)[0]
    sentiment = {0: "Negative", 1: "Neutral", 2: "Positive"}[prediction]
    print(f"{text} -> {sentiment}")
```

Output:

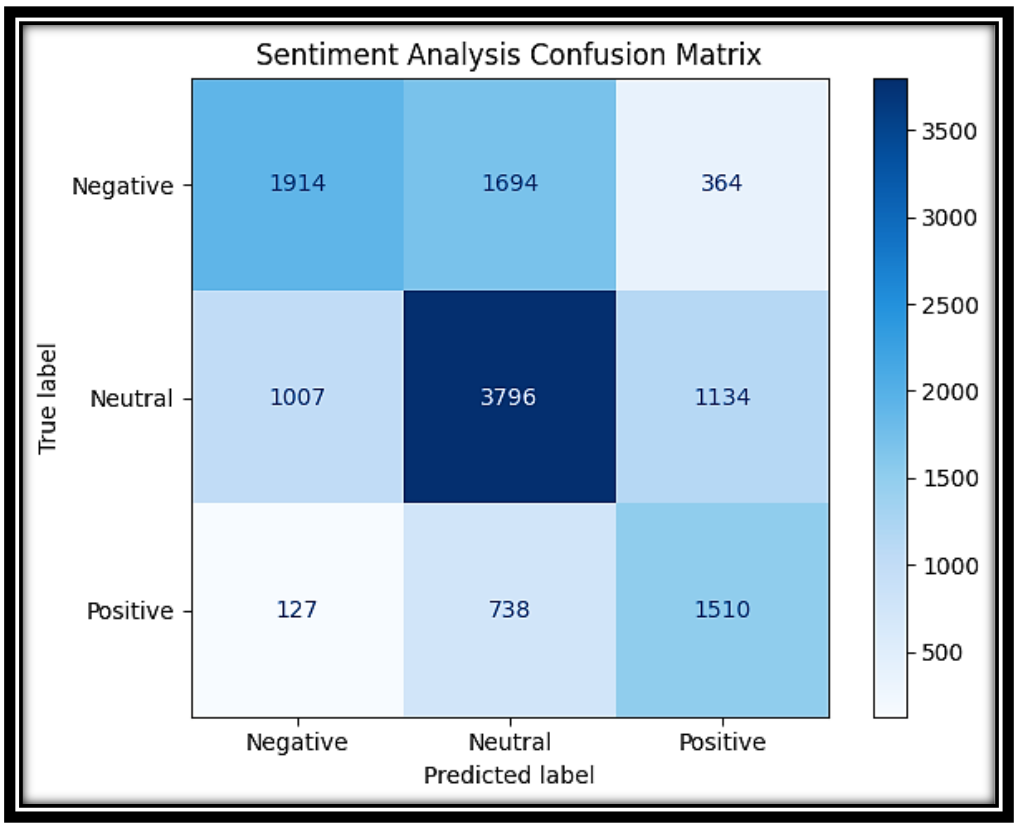
```
NLP SENTIMENT ANALYSIS
----------------------------------------
Training samples: 45615
Testing samples: 12284
Accuracy: 58.78 %
Precision: 0.5947
Recall: 0.5878
F1 Score: 0.5864

Classification Report:

              precision    recall  f1-score   support

    Negative       0.63      0.48      0.55      3972
     Neutral       0.61      0.64      0.62      5937
    Positive       0.50      0.64      0.56      2375

    accuracy                           0.59     12284
   macro avg       0.58      0.59      0.58     12284
weighted avg       0.59      0.59      0.59     12284
```



Learning Outcome:

PRACTICAL NO. 10

Aim: Write a program to analyse the resulting customer segments based on demographic, behavioural, or transactional characteristics to identify distinct customer groups and target them with tailored marketing strategies.

Theory:

Introduction to customer segmentation: Customer segmentation is the process of dividing customers into smaller groups based on similarities in their demographic, behavioural, or transactional characteristics. In Predictive Business Analytics, segmentation helps organisations understand different types of customers and develop effective marketing strategies. Customers may differ in age, income, purchasing frequency, spending habits, and product preferences. Treating all customers in the same way may lead to ineffective marketing campaigns and wasted resources. By identifying meaningful customer groups, businesses can personalise offers, improve customer satisfaction, increase customer retention, and allocate marketing budgets more efficiently.

Importance of customer data: Customer segmentation relies on relevant information that describes customer characteristics and behaviour. Demographic variables may include age and income, while behavioural and transactional variables can include purchase frequency, total spending, and product preferences. Analysing these characteristics helps businesses identify customers with similar needs and spending patterns. For example, some customers may have high incomes but low spending scores, while others may spend frequently or show strong interest in products. Understanding these differences allows organisations to design targeted campaigns, recommend suitable products, and develop customer loyalty programmes based on the characteristics of each group.

K-Means clustering: K-Means is an unsupervised machine learning algorithm used to divide data into a specified number of clusters. It works by assigning each data point to the nearest cluster centre, called a centroid, and repeatedly updating the centroids until the assignments stabilise or a stopping condition is reached. Unlike supervised learning, K-Means does not require predefined customer labels. Instead, it discovers groups from the patterns present in the data. Before clustering, numerical features are commonly standardised so that variables with larger numerical scales do not dominate the distance calculations.

Choosing and evaluating customer segments: The number of clusters influences the quality and usefulness of customer segmentation. The Elbow Method compares the within-cluster sum of squares for different values of K and looks for a point where additional clusters provide diminishing improvements. The Silhouette Score provides another evaluation measure by considering how closely each customer matches their own cluster compared with other clusters. A higher silhouette score generally indicates better separation and cohesion. However, these measures should be considered alongside business interpretation, since the

most useful segmentation is one that produces groups that are understandable, distinct, and actionable.

Marketing applications: After clustering, businesses can analyse the average characteristics of each group to develop appropriate marketing strategies. High-income, high-spending customers may respond to premium products and exclusive rewards, while high-income, low-spending customers may benefit from personalised recommendations or introductory offers. Customers with lower incomes and spending scores may respond better to value packs, discounts, and budget-friendly products. These interpretations should be based on the observed cluster profiles rather than assumed customer behaviour. Customer segmentation helps businesses improve campaign targeting, strengthen customer relationships, and make more informed decisions about promotions, product recommendations, and marketing investments.

Dataset Description:

The Mall Customers dataset contains customer information commonly used for demonstrating clustering and customer segmentation. It includes customer identifiers, gender, age, annual income in thousands of dollars, and a spending score assigned on a scale from 1 to 100. Age and annual income represent demographic and financial characteristics, while the spending score provides an indicator of purchasing behaviour. The dataset contains approximately 200 customer records and is small enough to analyse efficiently in Google Colab. In this experiment, annual income and spending score are used as the primary clustering features, while age is included to help interpret the resulting groups. K-Means clustering identifies customers with similar characteristics, and the average values of each cluster are used to suggest suitable marketing strategies. Since the dataset is illustrative, the resulting segments demonstrate the segmentation process rather than representing a universal description of real-world customers.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

url = "https://raw.githubusercontent.com/selva86/datasets/master/Mall_Customers.csv"

try:
    df = pd.read_csv(url)
except Exception:
    url = "https://huggingface.co/SezarTheGreat/mall-customer-segmentation-kmeans/resolve/main/Mall_Customers.csv"
    df = pd.read_csv(url)
```

```
df.columns = df.columns.str.strip()
df = df.rename(columns={
    "Genre": "Gender",
    "Annual Income (k$)": "Annual_Income",
    "Spending Score (1-100)": "Spending_Score"})

features = ["Age", "Annual_Income", "Spending_Score"]
X = df[features].copy()
X = X.fillna(X.median())

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

inertias = []
scores = []

for k in range(2, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertias.append(km.inertia_)
    scores.append(silhouette_score(X_scaled, labels))

best_k = range(2, 11)[int(np.argmax(scores))]
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X_scaled)

print("CUSTOMER SEGMENTATION ANALYSIS")
print("-" * 45)
print("Total customers:", len(df))
print("Selected number of clusters:", best_k)
print("Silhouette Score:", round(silhouette_score(X_scaled, df["Cluster"]), 3))

print("\nCUSTOMER SEGMENT PROFILES")
profile = df.groupby("Cluster")[features].mean().round(2)
profile["Customer_Count"] = df["Cluster"].value_counts().sort_index()
print(profile)
```

```
print("\nMARKETING STRATEGIES")
for cluster, row in profile.iterrows():
    income = row["Annual_Income"]
    spending = row["Spending_Score"]

    if income >= profile["Annual_Income"].median() and spending >= profile["Spending_Score"].median():
        strategy = "Premium offers, loyalty rewards, and exclusive products"
    elif income >= profile["Annual_Income"].median() and spending < profile["Spending_Score"].median():
        strategy = "Personalised recommendations and targeted incentives"
    elif income < profile["Annual_Income"].median() and spending >= profile["Spending_Score"].median():
        strategy = "Affordable trending products, bundles, and loyalty points"
    else:
        strategy = "Value deals, discounts, and budget-friendly promotions"

    print(f"Cluster {cluster}: {strategy}")

plt.figure(figsize=(8, 5))
plt.plot(range(2, 11), inertias, marker="o")
plt.xlabel("Number of Clusters")
plt.ylabel("Within-Cluster Sum of Squares")
plt.title("Elbow Method")
plt.xticks(range(2, 11))
plt.grid(True, alpha=0.3)
plt.show()

plt.figure(figsize=(9, 6))
scatter = plt.scatter(
    df["Annual_Income"],
    df["Spending_Score"],
    c=df["Cluster"],
    cmap="viridis",
    s=65,
    alpha=0.85)
```

```
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Customer Segments")
plt.colorbar(scatter, label="Cluster")
plt.show()

plt.figure(figsize=(8, 5))
profile["Customer_Count"].plot(kind="bar")
plt.xlabel("Cluster")
plt.ylabel("Number of Customers")
plt.title("Customers in Each Segment")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

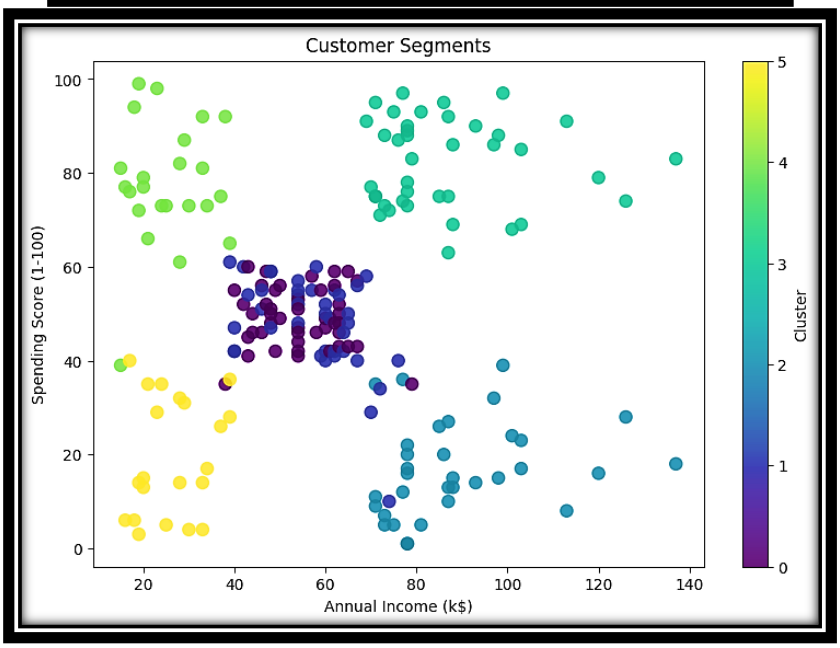
print("\nRESULT")
print(f"K-Means identified {best_k} customer segments based on age, annual income, and spending score.")
print("The cluster profiles and marketing suggestions can help businesses plan targeted campaigns.")
```

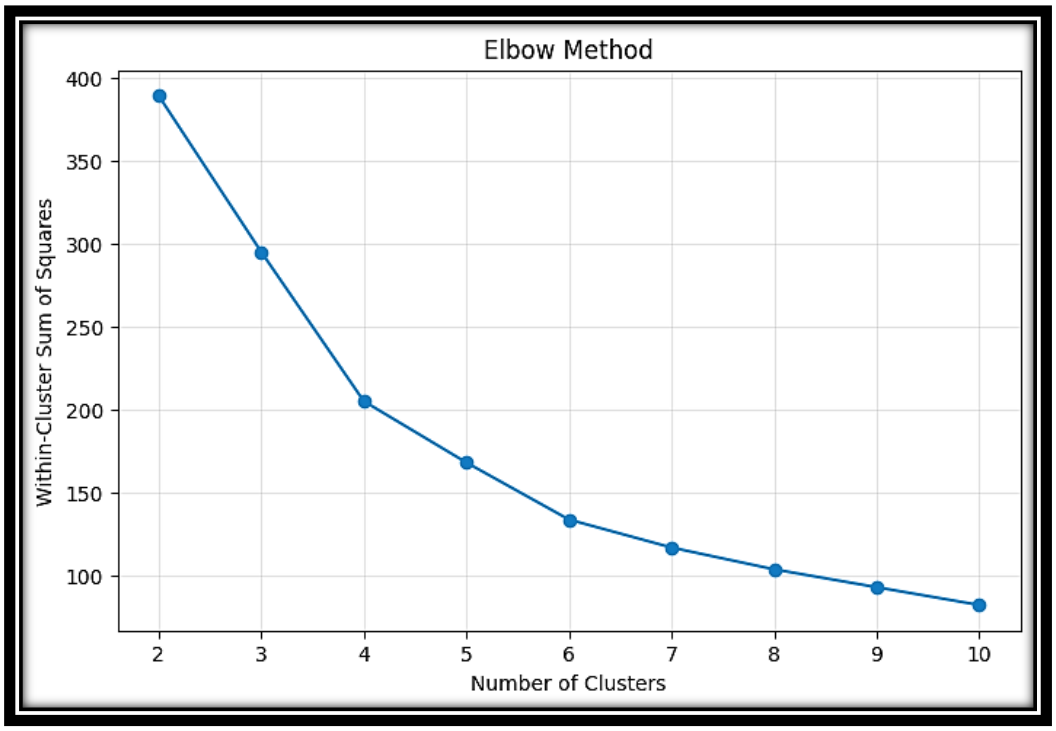
Output:

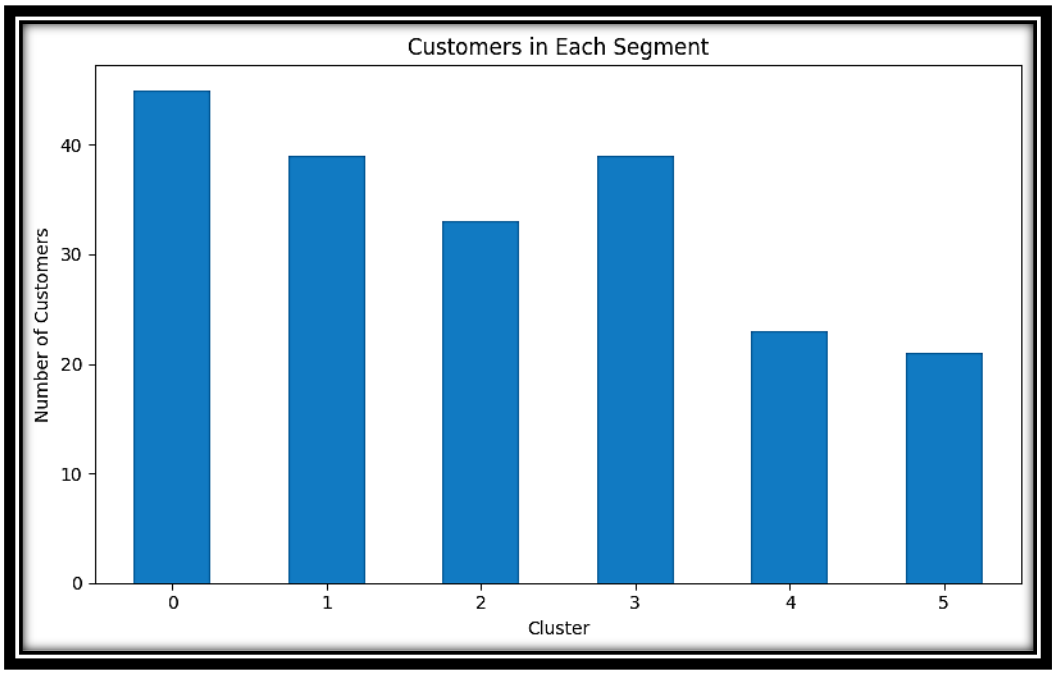
```
CUSTOMER SEGMENTATION ANALYSIS
----------------------------------------------
Total customers: 200
Selected number of clusters: 6
Silhouette Score: 0.428

CUSTOMER SEGMENT PROFILES
            Age  Annual_Income  Spending_Score  Customer_Count
Cluster
0         56.33          54.27           49.07              45
1         26.79          57.10           48.13              39
2         41.94          88.94           16.97              33
3         32.69          86.54           82.13              39
4         25.00          25.26           77.61              23
5         45.52          26.29           19.38              21

MARKETING STRATEGIES
Cluster 0: Affordable trending products, bundles, and loyalty points
Cluster 1: Personalised recommendations and targeted incentives
Cluster 2: Personalised recommendations and targeted incentives
Cluster 3: Premium offers, loyalty rewards, and exclusive products
Cluster 4: Affordable trending products, bundles, and loyalty points
Cluster 5: Value deals, discounts, and budget-friendly promotions
```







Learning Outcome: